



**Tecnológico
de Monterrey**

**MA2003B Aplicación de Métodos Multivariados
en Ciencia de Datos**

Módulo 1: Relaciones de Interdependencia

Profesor: Raul Gomez
Correo: rgomez@tec.mx
Oficina: A4-123A

August 29, 2025

Table of contents

I	Pruebas de Normalidad Univariada	3
1	Introducción al Tidyverse	5
1.1	Datos ordenados	5
1.1.1	Tibbles y pipes	5
1.1.2	Ejemplo: Ordenando el conjunto de datos de la OMS	6
1.2	La “Gramática de los Gráficos” y ggplot2	9
1.2.1	Layers in ggplot2	9
1.2.2	Graficación con ggplot2	10
1.3	Actividad: Limpieza de datos	11
2	Los momentos estandarizados	15
2.1	Asimetría	15
2.2	Curtosis	18
2.3	Gráficos QQ	20
2.4	Asimetría y curtosis muestrales	24
2.5	Escalamiento y estandarización	25
2.5.1	Escalamiento de variables aleatorias	26
2.5.2	Escalamiento de muestras	29
2.6	Actividad: Visualizando la no normalidad de muestras	34
3	Pruebas de normalidad	35
3.1	Pruebas de normalidad	35
3.2	Transformaciones	36
3.3	Optimización de normalidad (Box–Cox)	41
II	Análisis de Componentes Principales	45
4	Introducción al análisis multivariante	47
4.1	El vector de medias y la matriz de covarianzas	47
4.2	La distancia de Mahalanobis	49
4.3	Escalamiento y estandarización	55
4.4	Actividad: La geometría de los datos multivariantes	55
5	Distribución normal multivariante	57

5.1	Definición y propiedades básicas	57
5.2	Contornos de la normal multivariante	58
5.3	Estandarización de la normal multivariante	60
5.4	Geometric Interpretation of the SVD	61
5.5	Clausura bajo transformaciones lineales	64
5.6	Pruebas de normalidad multivariante	65
6	Análisis de componentes principales	67
6.1	Entendiendo la varianza total	67
6.2	Varianza acumulada y gráficos scree	72
6.3	Cargas (loadings) y biplots	76
6.4	Resumen del proceso de PCA	80
III	Análisis Factorial	83
7	Análisis factorial: modelos teóricos	85
7.1	Planteamiento del análisis factorial	85
7.2	Descomposición de varianza	86
7.3	Estimación de puntuaciones factoriales	87
7.4	Rotación de factores	87
8	Análisis factorial: métodos prácticos	89
8.1	Evaluación de la factorizabilidad	89
8.2	Decidir el número de factores	90
8.3	Extracción de factores	90
8.4	Rotación de factores	90
8.5	Computing Factor Scores	90
8.6	Assessing Model Fit	91
8.7	Visualizing the Model	91
8.8	Example: Factor Analysis of the Holzinger-Swineford Dataset	92
8.8.1	Assessing Factorability	92
8.8.2	Decidir el número de factores	94
8.8.3	Extracting, Rotating, and Scoring Factors	96
8.8.4	Assessing Model Fit	97
8.8.5	Visualizing the Model	98

Part I

Pruebas de Normalidad Univariada

Chapter 1

Introducción al Tidyverse

El Tidyverse es una colección de paquetes de R diseñados para la ciencia de datos. Proporciona una interfaz coherente y amigable para la manipulación, visualización y análisis de datos. Aunque se introdujo formalmente como ecosistema en 2016, muchos de estos paquetes fueron desarrollados por Hadley Wickham entre 2007 y 2014.

1.1 Datos ordenados

La idea central detrás del diseño del Tidyverse es la definición de “datos ordenados” de Wickham [3].

Definición 1.1.1. *Datos ordenados* es una forma de estructurar conjuntos de datos para facilitar su uso. En datos ordenados:

1. Cada variable forma una columna.
2. Cada observación forma una fila.
3. Cada tipo de unidad observacional forma una tabla.

Esta estructura facilita la manipulación, el análisis y la visualización de datos, ya que se alinea con la forma en que los datos se procesan típicamente en R. En esta sección describiremos los conceptos básicos del Tidyverse; para más información sobre su uso, el lector interesado puede consultar: [5, 6, 7].

1.1.1 Tibbles y pipes

En el Tidyverse, los datos suelen representarse mediante un tipo especial de data frame llamado *tibble*. Los tibbles son más amigables que los data frames tradicionales, pues ofrecen mejor impresión y comportamiento de subconjuntos. Además, están diseñados para trabajar de forma fluida con el operador pipe (`%>%`), que permite encadenar múltiples operaciones (conocidas como “verbos” en el contexto del Tidyverse) de manera clara y concisa. Para ilustrar estas ideas, en la siguiente sección realizaremos la limpieza del conjunto de datos de la OMS (“WHO”), para asegurar que el tibble asociado cumpla los principios de datos ordenados. Esto suele ser el primer paso del

Análisis Exploratorio de Datos (EDA) y es crucial para garantizar que los datos estén en un formato adecuado para su análisis y visualización.

1.1.2 Ejemplo: Ordenando el conjunto de datos de la OMS

Antes de comenzar, debemos cargar las librerías necesarias y el conjunto de datos de la OMS:

```
library("tidyverse")
library("here")
library("cowplot")
library("patchwork")
library("krulRutils")
library("ISLR2")
library("magrittr")

options(scipen = 999) # Desactivar notación científica
data(who)
```

Ahora podemos iniciar nuestro proceso de “ordenamiento”. El principal problema de este conjunto es que contiene variables (como “new_sp_m014”) que no son muy claras y que incluyen más de una pieza de información. Por ello necesitamos separar estas variables e introducir mejores nombres para ellas y sus valores asociados.

```
# Empezamos creando la variable "who_tidy"
# que contiene el conjunto de datos de la OMS ya limpio
who_tidy <- who %>%
  # Usamos pivot_longer para eliminar las variables que empiezan con "new"
  # y usarlas como valores en su lugar
  pivot_longer(
    cols = starts_with("new"),
    names_to = "key",
    values_to = "cases",
    values_drop_na = TRUE
  ) %>%
  mutate(
    key = if_else(
      startsWith(key, "newrel"),
      sub("newrel", "new_rel", key),
      key
    ),
    cases = as.integer(cases)
  ) %>%
  separate(key, into = c("new", "type", "sexage"), sep = "_") %>%
  separate(sexage, into = c("sex", "age"), sep = 1) %>%
  select(-new, -iso2, -iso3) %T>%
# Finalmente, guardamos los datos ordenados en formato RDS y CSV
saveRDS(here("data", "who_tidy.rds")) %>%
```

```
write_csv(here("data", "who_tidy.csv")) %>%
  glimpse()
```

Rows: 76,046

Columns: 6

```
$ country <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "A~
$ year    <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 19~
$ type    <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "s~
$ sex     <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f", "f~
$ age     <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014", "1~
$ cases   <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128, 90~
```

Ahora queremos convertir las columnas “type”, “sex” y “age” en factores. Empezamos construyendo las siguientes tablas de correspondencia:

```
who_type_lookup_tbl <- tibble(
  code = c("ep", "rel", "sn", "sp"),
  label = c(
    "Tuberculosis extrapulmonar",
    "Caso de recaída",
    "Tuberculosis pulmonar baciloscopía negativa",
    "Tuberculosis pulmonar baciloscopía positiva"
  )
)

who_sex_lookup_tbl <- tibble(
  code = c("f", "m"),
  label = c("Mujer", "Hombre")
)

who_age_lookup_tbl <- tibble(
  code = c(
    "014",
    "1524",
    "2534",
    "3544",
    "4554",
    "5564",
    "65"
  ),
  label = c(
    "0-14",
    "15-24",
    "25-34",
    "35-44",
    "45-54",
```

```
    "55-64",  
    "65+"  
  )  
)
```

Ahora podemos añadir las columnas factor para las variables correspondientes en el conjunto de la OMS:

```
who_factor <- who_tidy %>%  
  convert_codes_to_factor(  
    code_col = type,  
    lookup_tbl = who_type_lookup_tbl,  
    lookup_code_col = code,  
    lookup_label_col = label,  
  ) %>%  
  convert_codes_to_factor(  
    code_col = sex,  
    lookup_tbl = who_sex_lookup_tbl,  
    lookup_code_col = code,  
    lookup_label_col = label,  
  ) %>%  
  convert_codes_to_factor(  
    code_col = age,  
    lookup_tbl = who_age_lookup_tbl,  
    lookup_code_col = code,  
    lookup_label_col = label,  
  ) %>%  
  glimpse()
```

Rows: 76,046

Columns: 9

```
$ country    <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan"~  
$ year       <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997~  
$ type       <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp"~  
$ sex        <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f"~  
$ age        <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014"~  
$ cases      <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128~  
$ type_factor <fct> Tuberculosis pulmonar baciloscopía positiva, Tuberculosis ~  
$ sex_factor  <fct> Hombre, Hombre, Hombre, Hombre, Hombre, Hombre, Hombre, Mu~  
$ age_factor  <fct> 0-14, 15-24, 25-34, 35-44, 45-54, 55-64, 65+, 0-14, 15-24,~
```

Finally, we want to use this information to analyze the number of cases of tuberculosis by type and sex. We start by generating the corresponding frequency tibble.

```
who_type_sex_tbl <- who_factor %>%  
  count(type_factor, sex_factor, wt = cases, name = "cases") %>%  
  print(n = Inf)
```

```
# A tibble: 8 x 3
  type_factor          sex_factor    cases
  <fct>              <fct>      <int>
1 Tuberculosis extrapulmonar      Mujer    941880
2 Tuberculosis extrapulmonar      Hombre  1044299
3 Caso de recaída                Mujer    1201596
4 Caso de recaída                Hombre   2018976
5 Tuberculosis pulmonar baciloscopia negativa Mujer    2439139
6 Tuberculosis pulmonar baciloscopia negativa Hombre   3840388
7 Tuberculosis pulmonar baciloscopia positiva Mujer    11324409
8 Tuberculosis pulmonar baciloscopia positiva Hombre   20586831
```

1.2 La “Gramática de los Gráficos” y ggplot2

El concepto de la “Gramática de los Gráficos” fue introducido por Leland Wilkinson en su libro de 1999 *The Grammar of Graphics* [8]. Proporciona un marco para entender cómo crear visualizaciones de forma sistemática. Descompone el proceso de construir una gráfica en componentes como datos, estéticas, geometrías, estadísticas, coordenadas y temas. El paquete **ggplot2** [4] implementa esta gramática en R, permitiendo construir visualizaciones complejas por capas. Para más información sobre su historia y aplicaciones, consulte: [1, 2].

1.2.1 Layers in ggplot2

In **ggplot2**, plots are constructed by adding multiple *layers* that define the components of the visualization. Each layer corresponds to a specific aspect of the plot, and together they form the complete graphic. These layers include:

1. **Data:** The dataset to be visualized (usually a data frame or tibble). This is the source of the information for the plot.
2. **Aesthetics:** Mappings that relate variables in the data to visual properties of the plot, such as:
 - Position on the x- and y-axes.
 - Color, fill.
 - Size, shape.
 - Transparency.

These mappings define *what* data is shown and *how* it is represented visually.

3. **Geometries:** Geometric objects that display the data, for example:
 - `geom_point()` for scatterplots.
 - `geom_line()` for line graphs.
 - `geom_col()` for bar charts.
 - `geom_histogram()` for histograms.

The geometry controls *how* the data is drawn.

4. **Statistical transformations:** Optional calculations applied to the data before plotting, such as:
 - `stat_bin()` for binning data in histograms.
 - `stat_smooth()` for fitting smooth curves.

These are preprocessing steps for the data visualization.

5. **Scales:** Define how data values are translated into visual properties, for example mapping numeric values to colors or shapes. Scales also control axis ticks, legends, and guides.
6. **Coordinates:** The coordinate system used for the plot, such as Cartesian (`coord_cartesian()`), polar (`coord_polar()`), or flipped coordinates (`coord_flip()`).
7. **Facets:** Methods to split the data into subsets and display multiple plots arranged in a grid:
 - `facet_wrap()`
 - `facet_grid()`
8. **Labels:** `labs()` adds titles, axis labels, and captions.
9. **Themes:** `theme()` controls the overall appearance of the plot (fonts, background, gridlines).

Each layer can be combined and customized to build complex and elegant plots. This *layered grammar* allows you to think of your graphic as a composition of independent components rather than a single, monolithic object.

1.2.2 Graficación con ggplot2

Ilustraremos estos conceptos graficando el número de casos de tuberculosis por tipo y sexo, usando el tibble de frecuencias que creamos antes.

```
who_type_sex_plot <- who_type_sex_tbl %>%  
  ggplot(aes(x = type_factor, y = cases, fill = sex_factor)) +  
  geom_col(position = "dodge") +  
  c_scale_fill("C rose", "C blue") +  
  labs(  
    title = "Casos de tuberculosis por tipo y sexo",  
    x = "Tipo de tuberculosis",  
    y = "Casos",  
    fill = "Sexo"  
  ) +  
  theme_krul()
```

Ahora podemos guardar y graficar:

```
ggsave(  
  filename = here("images", "who_plot.png"),  
  plot = who_type_sex_plot,
```

```
width = 12,
height = 6,
dpi = 300
)
who_type_sex_plot
```

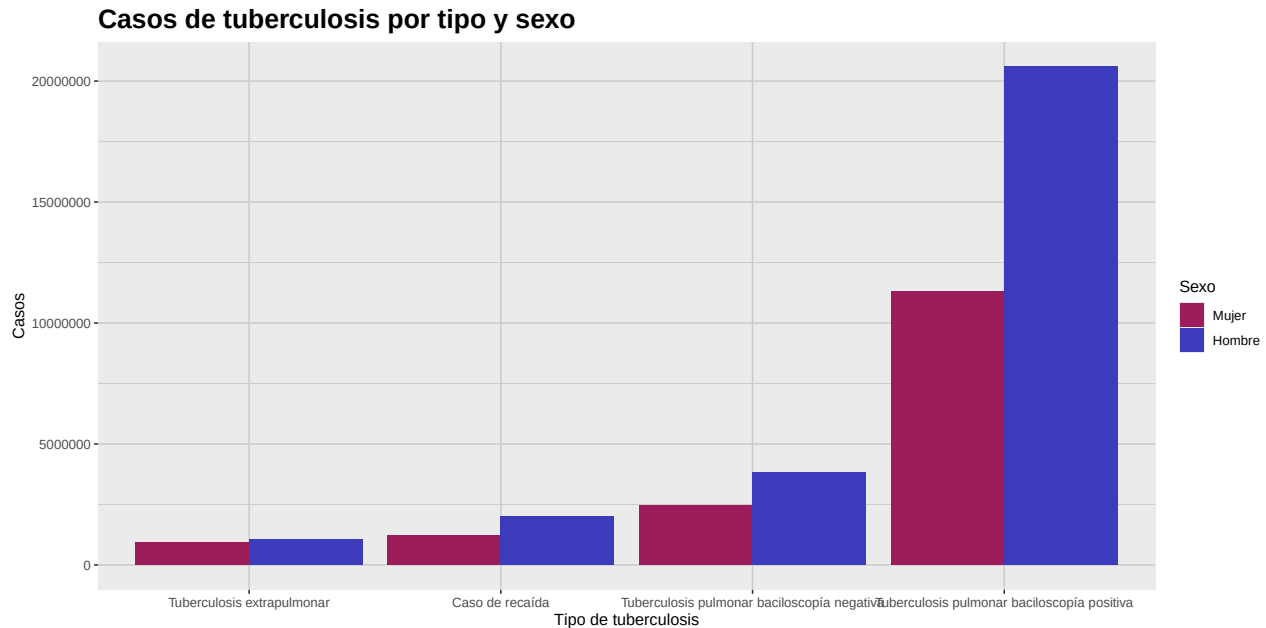


Figure 1.1: Esta gráfica muestra un análisis comparativo del número de casos de tuberculosis por tipo y sexo. Como se observa, el número de casos es consistentemente mayor en la población masculina en todos los tipos de tuberculosis.

1.3 Actividad: Limpieza de datos

1. En la sección anterior se mostró cómo usar el operador `%>%` para encadenar múltiples operaciones en el Tidyverse. En este ejercicio, muestre el resultado de todos los pasos intermedios en el proceso de ordenamiento del conjunto de la OMS, usando el operador `pipe`.
2. El problema con el conjunto *Billboard*, según los principios de "datos ordenados", es que las semanas en que una canción apareció en la lista están representadas como columnas, lo que dificulta analizar los datos. Utilizando las técnicas vistas hasta ahora, ordene el conjunto para analizar la evolución del ranking de la canción "Who Let The Dogs Out" de "Baha Men" en el Billboard Hot 100. La visualización final debe lucir como la que se muestra abajo.

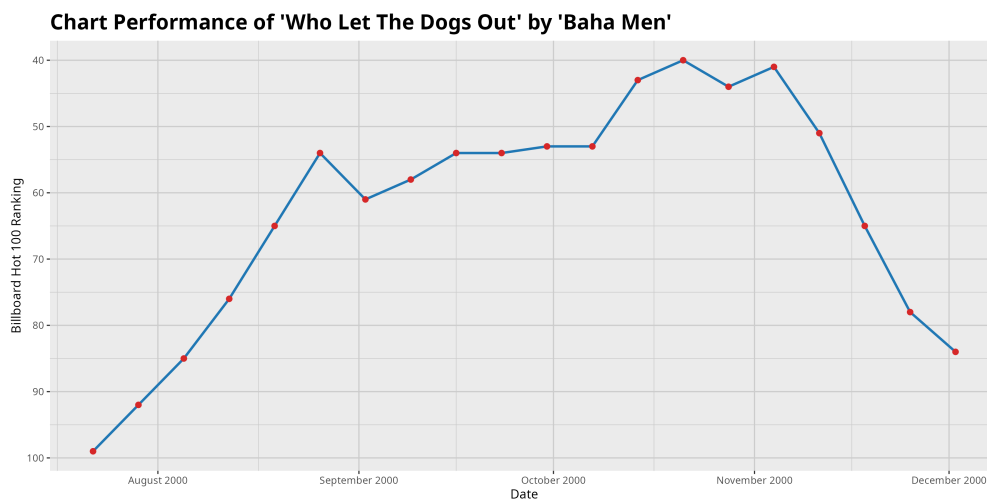


Figure 1.2: Evolución del ranking de la canción 'Who Let The Dogs Out' de 'Baha Men' en el Billboard Hot 100. La canción alcanzó el puesto 40 en la semana del 21 de octubre de 2000 y permaneció varias semanas en el top 100.

Bibliography

- [1] Cleveland, W. S. (1993) *The Elements of Graphing Data*. Hobart Press.
- [2] Hyndman, R. J., and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- [3] Wickham, H. (2014) Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10>
- [4] Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag.
- [5] Wickham, H., and Grolemund, G. (2017) *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O’Reilly Media.
- [6] Wickham, H. (2019) Functional programming with purrr. *UseR! Conference*.
- [7] Wickham, H., et al. (2019) Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- [8] Wilkinson, L. (1999) *The Grammar of Graphics*. Springer-Verlag.

Chapter 2

Los momentos estandarizados

Como es bien sabido, cualquier distribución normal queda determinada por su media y desviación estándar. Sin embargo, muchos conjuntos de datos reales no siguen una distribución normal. En este capítulo exploraremos otras distribuciones y mostraremos cómo los conceptos de asimetría (skewness) y curtosis ayudan a entender su forma. También introduciremos los gráficos QQ, herramientas útiles para comparar la distribución de un conjunto de datos con una distribución teórica (usualmente la normal). Antes de comenzar, carguemos las librerías necesarias y fijemos algunas opciones.

```
library(tidyverse) # for data manipulation and visualization
library(broom) # for tidying data
library(here) # for file paths
library(patchwork) # for combining ggplots
library(krulRutils) # for custom package functions
library(magrittr) # for the pipe operator
library(e1071) # for skewness and kurtosis functions
library(car) # for robust scaling functions
library(ggforce) # for ggplot2 extensions
library(mvtnorm) # for multivariate normal distribution
library(MVN) # for multivariate normality tests
library(ellipse) # for generating covariance ellipses
library(ggrepel) # for better text annotations in ggplot2

options(scipen = 999)
```

2.1 Asimetría

Definición 2.1.1. La *asimetría* de una variable aleatoria X se define como el tercer momento estandarizado, dado por

$$\gamma_1 = \frac{\mu_3}{\sigma^3} = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3},$$

donde $\mu = \mathbb{E}[X]$ es la media y $\sigma = \sqrt{\mathbb{E}[(X - \mu)^2]}$ es la desviación estándar de X .

La asimetría mide la falta de simetría de una distribución. Una distribución es *positivamente asimétrica* (cola a la derecha) si posee una cola más larga a la derecha, y *negativamente asimétrica* (cola a la izquierda) si la cola más larga está a la izquierda. Una distribución perfectamente simétrica tiene asimetría cero. Para ilustrar estas ideas, introduciremos la distribución Gamma.

Definición 2.1.2. Decimos que una variable aleatoria continua X sigue una *distribución Gamma* con parámetro de forma $\alpha > 0$ y parámetro de tasa $\beta > 0$, si su función de densidad de probabilidad (PDF) está dada por

$$f_X(x; \alpha, \beta) = \begin{cases} \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, & x > 0, \\ 0, & x \leq 0, \end{cases} \quad (2.1)$$

donde $\Gamma(\alpha)$ denota la función Gamma, definida como

$$\Gamma(\alpha) = \int_0^\infty t^{\alpha-1} e^{-t} dt.$$

Entre otras aplicaciones, la distribución Gamma se usa frecuentemente para modelar el número total de reclamaciones de seguros en un periodo dado.

Comentario 2.1.3. La asimetría de una distribución Gamma está dada por

$$\gamma_1 = \frac{2}{\sqrt{\alpha}}.$$

Esto implica que la asimetría es siempre positiva y tiende a cero conforme α aumenta; es decir, la distribución se vuelve más simétrica al crecer α .

Mostraremos cómo graficar la PDF de la distribución Gamma para $\alpha = 2$ y $\beta = 1$, destacando la media y la mediana.

```
# Parameters
alpha <- 2
beta <- 1

# Create sequence of x values
x_data <- seq(0, 10, length.out = 200)

# Calculate density values
gamma_distribution_tbl <- tibble(
  x_data = x_data,
  density = dgamma(x_data, shape = alpha, rate = beta)
)

# Calculate mean and median
gamma_mean <- alpha / beta
gamma_median <- qgamma(0.5, shape = alpha, rate = beta)
```

```
gamma_sd <- sqrt(alpha) / beta

# Plot
gamma_distribution_plot <- gamma_distribution_tbl %>%
  ggplot(aes(x = x_data, y = density)) +
  geom_line(
    color = c_palette["C blue"],
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_mean,
    color = c_palette["C red"],
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_median,
    color = c_palette["C green"],
    linetype = "dotted",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = gamma_mean,
    y = 0.5,
    label = "Media",
    color = c_palette["C red"],
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = gamma_median,
    y = 0.5,
    label = "Mediana",
    color = c_palette["C green"],
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.6)) +
  labs(
    title = paste(
      "PDF de la distribución Gamma (shape =", alpha, ", rate =", beta, ")"
    ),
    x = "X",
    y = "Densidad"
  ) +
```

```
theme_krul()
```

```
gamma_distribution_plot
```

PDF de la distribución Gamma (shape = 2 , rate = 1)

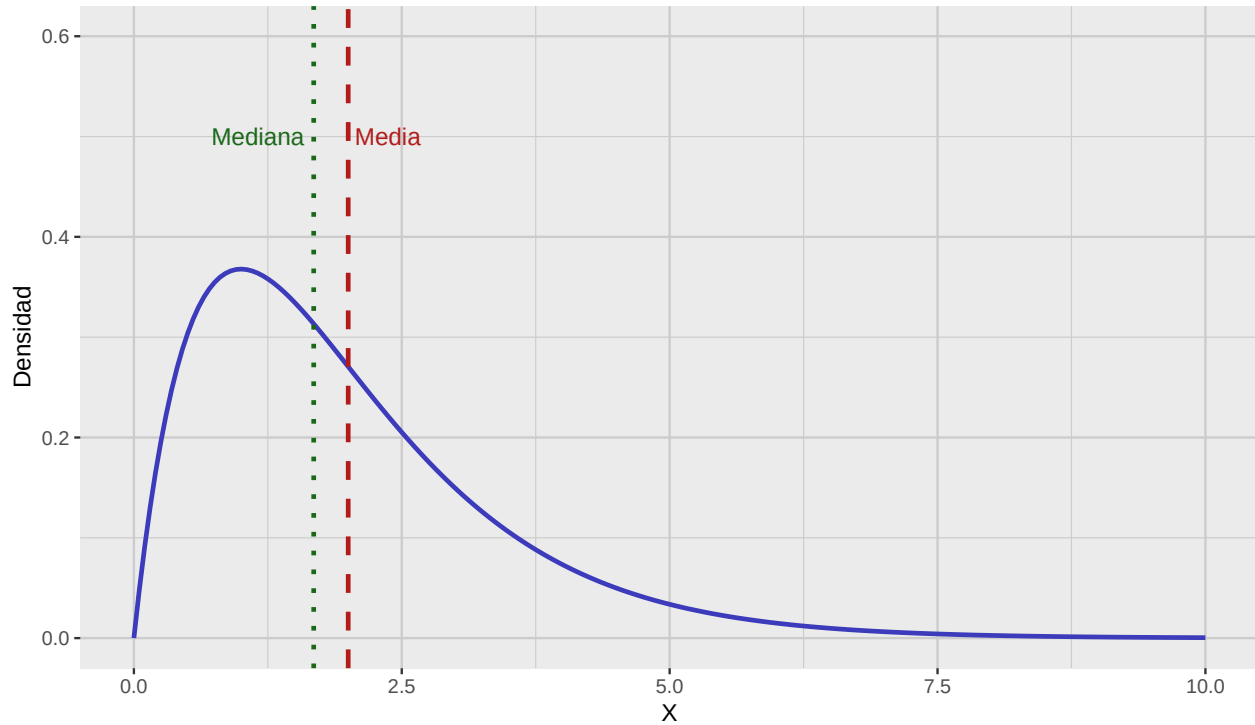


Figure 2.1: Función de densidad (PDF) de la distribución gamma con media y mediana destacadas.

2.2 Curtosis

Definición 2.2.1. La *curtosis* de una variable aleatoria X se define como el cuarto momento estandarizado, dado por

$$\beta_2 = \frac{\mu_4}{\sigma^4} = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4},$$

donde $\mu = \mathbb{E}[X]$ es la media y $\sigma = \sqrt{\mathbb{E}[(X - \mu)^2]}$ es la desviación estándar de X .

La curtosis mide el grosor de las colas de una distribución. Una distribución con alta curtosis (*leptocúrtica*) tiene colas más pesadas y un pico más agudo que la normal; con baja curtosis (*platicúrtica*) tiene colas más ligeras y un pico más plano. La curtosis de la normal es 3; por ello suele calcularse la *curtosis en exceso* dada por:

$$\gamma_2 = \beta_2 - 3.$$

Para ilustrar, introducimos la distribución de Laplace.

Definición 2.2.2. Decimos que una variable aleatoria continua X sigue una *distribución de Laplace* con parámetro de localización $\mu \in \mathbb{R}$ y de escala $b > 0$, si su PDF está dada por

$$f_X(x; \mu, b) = \frac{1}{2b} \exp\left(-\frac{|x - \mu|}{b}\right), \quad x \in \mathbb{R}. \quad (2.2)$$

Entre otras aplicaciones, la distribución de Laplace se usa en procesamiento de imágenes y visión por computadora, particularmente para modelar ruido.

Comentario 2.2.3. La curtosis en exceso de una distribución de Laplace es

$$\gamma_2 = 3.$$

Esto significa que la distribución de Laplace tiene colas más pesadas que la normal.

Mostramos ahora cómo graficar la PDF de la distribución de Laplace con media cero y varianza uno.

```
# Parameters
mu <- 0 # mean (location)
b <- 1 / sqrt(2) # scale

# Sequence of x values covering enough range to see tails
x_data <- seq(-5, 5, length.out = 300)

# Laplace PDF function
dlaplace <- function(x, mu, b) {
  1 / (2 * b) * exp(-abs(x - mu) / b)
}

# Create data frame with PDF values
laplace_distribution_tbl <- tibble(
  x_data = x_data,
  density = dlaplace(x_data, mu, b)
)

# Plot PDF
laplace_distribution_tbl %>%
  ggplot(aes(x = x_data, y = density)) +
  geom_line(color = c_palette["C blue"], linewidth = 1) +
  labs(
    title = "PDF de la distribución de Laplace (media = 0, var = 1)",
    x = "x",
    y = "Densidad"
  ) +
  theme_krul()
```

PDF de la distribución de Laplace (media = 0, var = 1)

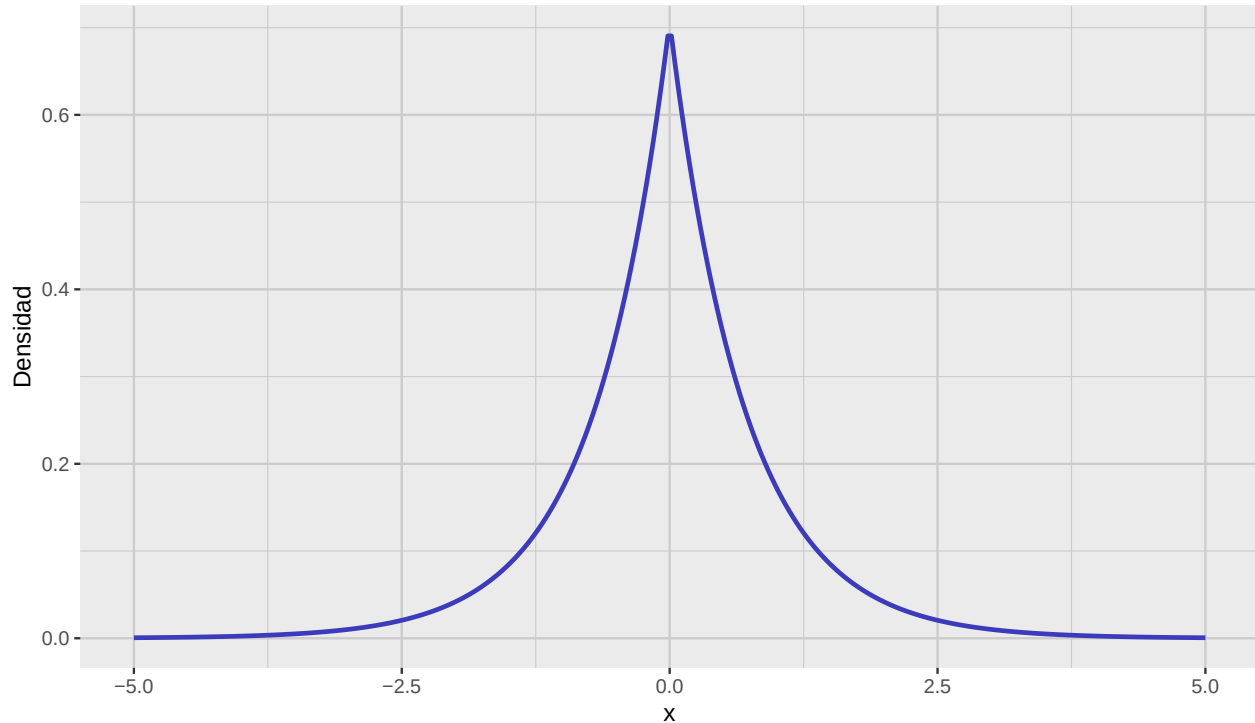


Figure 2.2: PDF de la distribución de Laplace con media cero y varianza uno.

2.3 Gráficos QQ

Los gráficos QQ (quantile–quantile) son herramientas para comparar la distribución de un conjunto de datos con una distribución teórica (por ejemplo, la normal). Son especialmente útiles para evaluar la normalidad.

Para construir un gráfico QQ, siga estos pasos:

1. **Ordene los datos muestrales:** Coloque los datos en orden creciente:

$$x_{(1)} \leq x_{(2)} \leq \cdots \leq x_{(n)}.$$

2. **Calcule posiciones de trazado (cuantiles empíricos):** Para cada valor ordenado $x_{(i)}$, calcule la probabilidad correspondiente

$$p_i = \frac{2i - 1}{2n}, \quad i = 1, 2, \dots, n.$$

3. **Obtenga los cuantiles teóricos de la normal:** A partir de la función cuantil (inversa de la CDF) de la normal estándar:

$$q_i = \Phi^{-1}(p_i),$$

donde Φ^{-1} es la función cuantil de la normal estándar.

4. **Trace los puntos:** Grafique los pares $(q_i, x_{(i)})$ con q_i en el eje x y $x_{(i)}$ en el eje y.
5. **Agregue una línea de referencia:** Trazar, por ejemplo, una recta por el primer y tercer cuartil ayuda a evaluar la normalidad.

Mientras más cerca caigan los puntos de esa recta, más se parece la muestra a la distribución normal especificada.

Para ilustrar estas ideas, generaremos muestras de las distribuciones Gamma y Laplace y construiremos sus gráficos QQ e histogramas. Primero, generamos las muestras:

```
set.seed(123) # for reproducibility

# Sample size
n <- 300
# Generate random samples from the Gamma distribution
gamma_sample <- rgamma(n, shape = alpha, rate = beta)

gamma_sample_tbl <- tibble(
  sample = gamma_sample
)

# Generate uniform random numbers in (0,1)
u <- runif(n)

# Apply inverse CDF of Laplace
laplace_sample <- ifelse(u < 0.5,
  mu + b * log(2 * u),
  mu - b * log(2 * (1 - u))
)

laplace_sample_tbl <- tibble(
  sample = laplace_sample
)
```

Ahora construiremos los correspondientes gráficos QQ e histogramas. Empezamos con las muestras de la distribución Gamma:

```
gamma_qq_plot <- gamma_sample_tbl %>%
  ggplot(aes(sample = sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
```

```
labs(
  title = "Gráfico QQ",
  x = "Cuantiles teóricos",
  y = "Cuantiles muestrales"
) +
theme_krul()

gamma_histogram <- gamma_sample_tbl %>%
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histograma",
    x = "Valores muestrales",
    y = "Densidad"
  ) +
  theme_krul()

gamma_plot <- gamma_qq_plot + gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Distribución Gamma: gráfico QQ e histograma",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

gamma_plot
```

Distribución Gamma: gráfico QQ e histograma

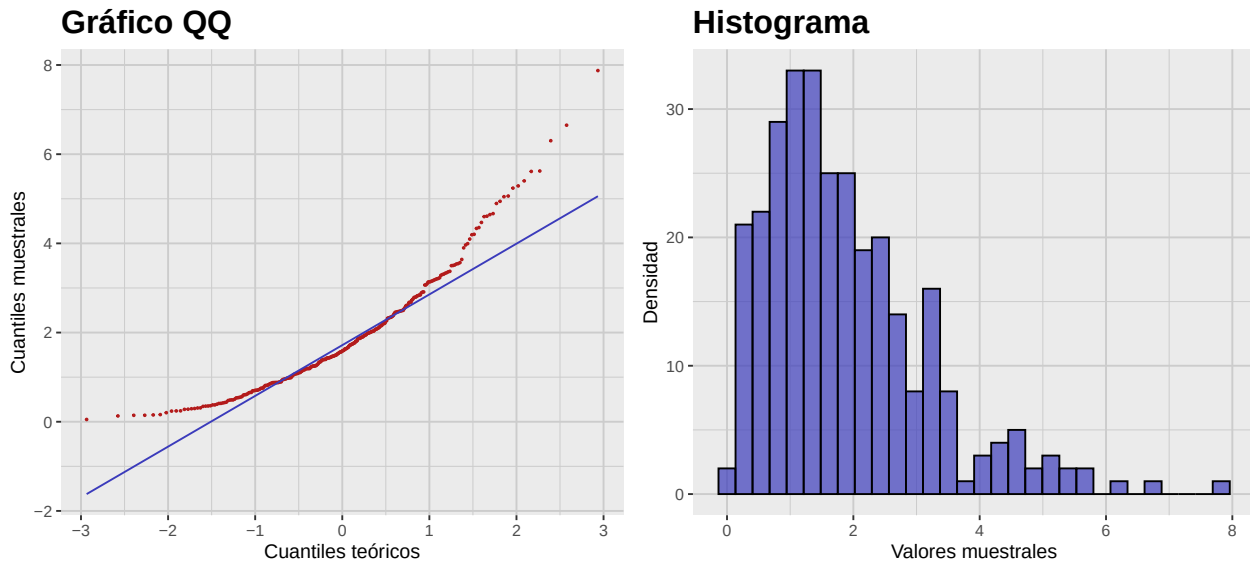


Figure 2.3: Gráfico QQ e histograma de las muestras de la distribución Gamma.

Construimos ahora el gráfico QQ y el histograma para las muestras de Laplace:

```
laplace_qq_plot <- laplace_sample_tbl %>%
  ggplot(aes(sample = sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "Gráfico QQ",
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

laplace_histogram <- laplace_sample_tbl %>%
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  )
```

```
) +
labs(
  title = "Histograma",
  x = "Valores muestrales",
  y = "Densidad"
) +
theme_krul()
laplace_plot <- laplace_qq_plot + laplace_histogram +
plot_layout(ncol = 2) +
plot_annotation(
  title = "Distribución de Laplace: gráfico QQ e histograma",
  theme = theme(
    plot.title = element_text(
      size = 24,
      face = "bold"
    )
  )
)
laplace_plot
```

Distribución de Laplace: gráfico QQ e histograma

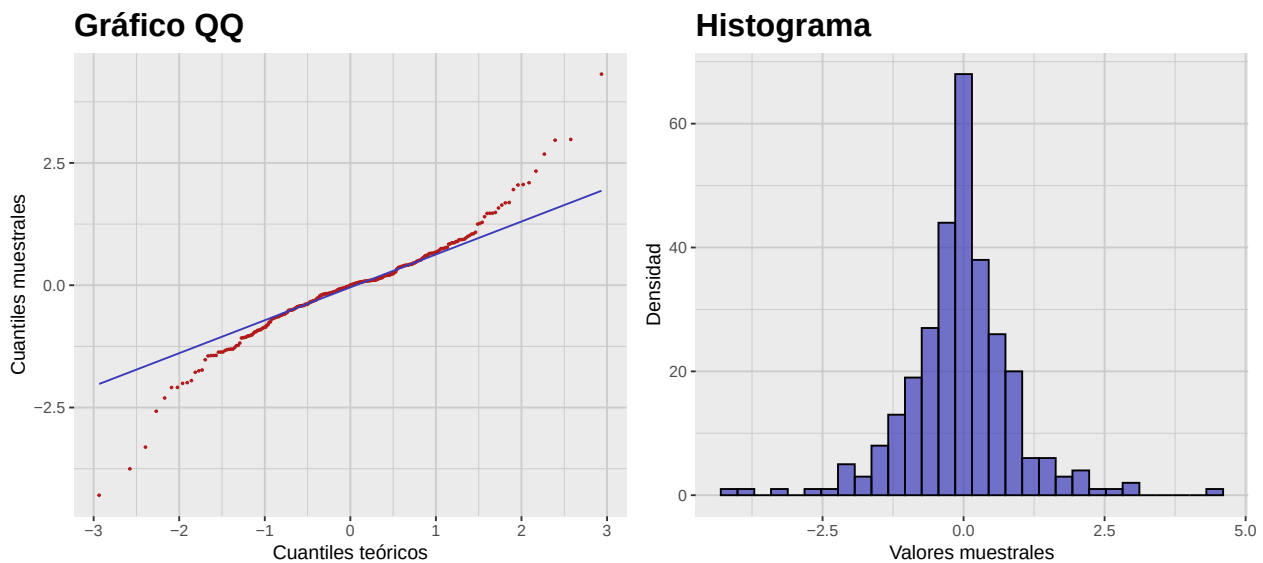


Figure 2.4: Gráfico QQ e histograma de las muestras de la distribución de Laplace.

2.4 Asimetría y curtosis muestrales

So far we have defined skewness and kurtosis for random variables. However, in practice, we often work with samples rather than entire populations. Therefore, we need to define sample skewness and sample kurtosis.

Definición 2.4.1. Let $x_1, x_2, \dots, x_n$ be a sample of size n . The *sample mean* $\bar{x}$ and *sample variance* s^2 are given by

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

The *sample skewness* g_1 is given by

$$g_1 = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^3.$$

The *sample excess kurtosis* g_2 is given by

$$g_2 = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}.$$

As an example, we will compute the sample skewness and sample excess kurtosis of the Gamma and Laplace samples we generated earlier. For the Gamma sample we have:

```
# Compute sample skewness and excess kurtosis for Gamma distribution
gamma_sample_tbl %>%
  summarise(
    sample_skewness = skewness(sample, type = 2),
    sample_excess_kurtosis = kurtosis(sample, type = 2)
  ) %>%
  glimpse()
```

```
Rows: 1
Columns: 2
$ sample_skewness      <dbl> 1.293772
$ sample_excess_kurtosis <dbl> 2.164353
```

Para la muestra de Laplace tenemos:

```
# Compute sample skewness and excess kurtosis for Laplace distribution
laplace_sample_tbl %>%
  summarise(
    sample_skewness = skewness(sample, type = 2),
    sample_excess_kurtosis = kurtosis(sample, type = 2)
  ) %>%
  glimpse()
```

```
Rows: 1
Columns: 2
$ sample_skewness      <dbl> -0.07876743
$ sample_excess_kurtosis <dbl> 3.726374
```

2.5 Escalamiento y estandarización

En muchas situaciones, las unidades con que medimos distintas variables no son comparables (p.ej., estatura en cm y peso en kg). En esos casos conviene *escalar* las variables para hacerlas

comparables. En esta sección discutimos los métodos más comunes para escalar y estandarizar variables y muestras.

2.5.1 Escalamiento de variables aleatorias

Definición 2.5.1. Una *transformación afín* de una variable aleatoria X es de la forma

$$Y = aX + b$$

donde a y b son constantes.

Comentario 2.5.2. En estadística, a menudo se habla de *escalamiento* (aunque incluya traslación) para referirse a transformaciones afines.

Suponga que X tiene media μ y desviación estándar σ . Entonces la variable

$$Y = aX + b$$

tendrá media $a\mu + b$ y desviación estándar $|a|\sigma$. En particular, si elegimos $a = 1/\sigma$ y $b = -\mu/\sigma$, la variable

$$Z = \frac{X - \mu}{\sigma}$$

tendrá media 0 y desviación estándar 1. Esta transformación se llama *estandarización*, y Z es la versión *estandarizada* de X (también llamada *puntaje z*).

Comentario 2.5.3. Si X no es normal, Z no será normal estándar. La estandarización no transforma una distribución no normal en normal.

Observación 2.5.4. Por definición,

$$\gamma_1(Z) = \mathbb{E}[Z^3] = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3} = \gamma_1(X)$$

y

$$\gamma_2(Z) = \mathbb{E}[Z^4] - 3 = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4} - 3 = \gamma_2(X).$$

Como ejemplo, consideremos de nuevo la distribución Gamma. En este caso

$$\mu = \frac{\alpha}{\beta} \quad \text{and} \quad \sigma = \frac{\sqrt{\alpha}}{\beta}.$$

Con esta información, generamos la gráfica de la versión estandarizada de la Gamma y la comparamos con la original.

```
# Calculate density values
z_gamma_distribution_tbl <- gamma_distribution_tbl %>%
  mutate(
    z_data = (x_data - gamma_mean) / gamma_sd,
    z_density = dgamma(x_data, shape = alpha, rate = beta) * gamma_sd
  ) %>%
  select(z_data, z_density)
```

```
# Calculate mean and median
z_gamma_mean <- 0
z_gamma_median <- (gamma_median - gamma_mean) / gamma_sd

# Plot
z_gamma_distribution_plot <- z_gamma_distribution_tbl %>%
  ggplot(aes(x = z_data, y = z_density)) +
  geom_line(
    color = c_palette["C blue"],
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_mean,
    color = c_palette["C red"],
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_median,
    color = c_palette["C green"],
    linetype = "dotted",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = z_gamma_mean,
    y = 0.55,
    label = "Media",
    color = c_palette["C red"],
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = z_gamma_median,
    y = 0.55,
    label = "Mediana",
    color = c_palette["C green"],
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.6)) +
  labs(
    title = paste(
      "Estandarizada"
    ),
  ),
```

```
x = "X",
y = "Densidad"
) +
theme_krul()

gamma_distribution_plot <- gamma_distribution_plot +
  labs(
    title = "Original"
  )

combined_gamma_plot <- gamma_distribution_plot + z_gamma_distribution_plot +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Distribución Gamma: original y estandarizada",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

combined_gamma_plot
```

Distribución Gamma: original y estandarizada

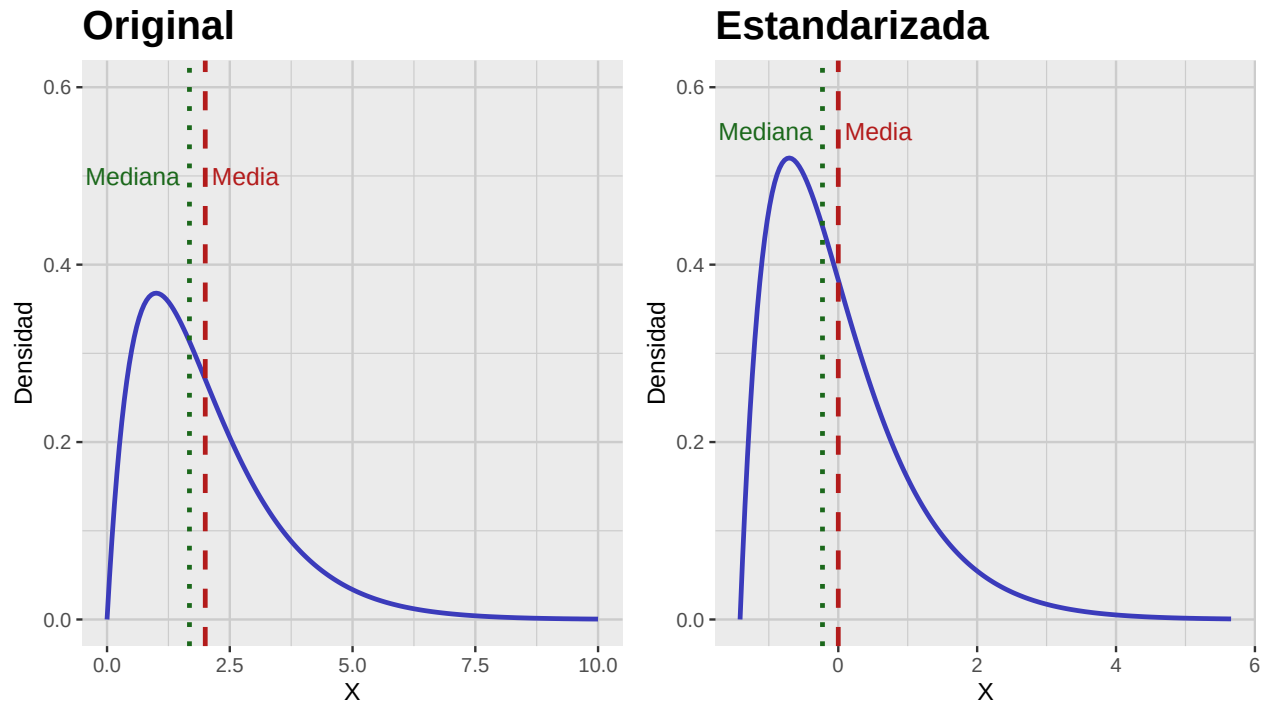


Figure 2.5: Comparación de la función de densidad (PDF) de la distribución Gamma original y su versión estandarizada.

2.5.2 Escalamiento de muestras

Para una muestra dada, no siempre se conoce la distribución subyacente; en particular, pueden desconocerse la media y la desviación poblacionales. Aun así, podemos escalar los datos; hay varios métodos comunes:

1. *Estandarización*: Usa la media y desviación muestrales. La muestra estandarizada es

$$T = \frac{X - \bar{X}}{S},$$

donde $\bar{X}$ es la media muestral y S la desviación muestral.

2. *Escalamiento Min-Max*: Escala a un rango fijo, usualmente $[0,1]$. La muestra escalada es

$$T = \frac{X - \min(X)}{\max(X) - \min(X)},$$

donde $\min(X)$ y $\max(X)$ son el mínimo y máximo de la muestra. Es muy sensible a atípicos al usar extremos.

3. *Escalamiento robusto*: Usa la mediana y el rango intercuartil (IQR). La muestra escalada es

$$T = \frac{X - \text{median}(X)}{\text{IQR}(X)},$$

donde $\text{IQR}(X) = Q_3 - Q_1$. Es más robusto a atípicos que Min-Max.

4. *Normalización*: Divide cada dato por la norma del vector. La muestra normalizada es

$$T = \frac{X}{\|X\|},$$

donde $\|X\|$ es la norma. Se usa con menor frecuencia que los anteriores.

Comentario 2.5.5. Como se mencionó, escalar no vuelve normal una distribución no normal. En particular, la asimetría y la curtosis en exceso muestrales no cambian.

As an example, we will consider the different scaling methods applied to the Gamma sample we generated earlier.

```
scale_lookup_tbl <- tibble(
  code = c(
    "sample",
    "sample_standard",
    "sample_min_max",
    "sample_robust",
    "sample_normal"
  ),
  label = c(
    "Original Sample",
    "Estandarizada",
    "Min-Max Scaled",
    "Robust Scaled",
    "Normalized"
  )
)

scaled_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    sample_standard = standard_scale(sample),
    sample_min_max = min_max_scale(sample),
    sample_robust = robust_scale(sample),
    sample_normal = normal_scale(sample)
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = "scaling_method",
    values_to = "sample_scaled"
  ) %>%
  convert_codes_to_factor(
    code_col = scaling_method,
    lookup_tbl = scale_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
```

```
    new_factor_col_name = scaling_method_factor
  )

scaled_gamma_qq_plot <- scaled_gamma_sample_tbl %>%
  ggplot(aes(sample = sample_scaled)) +
  facet_wrap(
    ~scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "Gráficos QQ",
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

scaled_gamma_histogram <- scaled_gamma_sample_tbl %>%
  ggplot(aes(x = sample_scaled)) +
  facet_wrap(
    ~scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogramas",
    x = "Valores muestrales",
    y = "Densidad"
  ) +
  theme_krul()
```

```
scaled_gamma_plot <- scaled_gamma_qq_plot + scaled_gamma_histogram +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Comparación de métodos de escalamiento aplicados a la muestra Gamma",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
scaled_gamma_plot
```

Comparación de métodos de escalamiento aplicados a la muestra Gamma

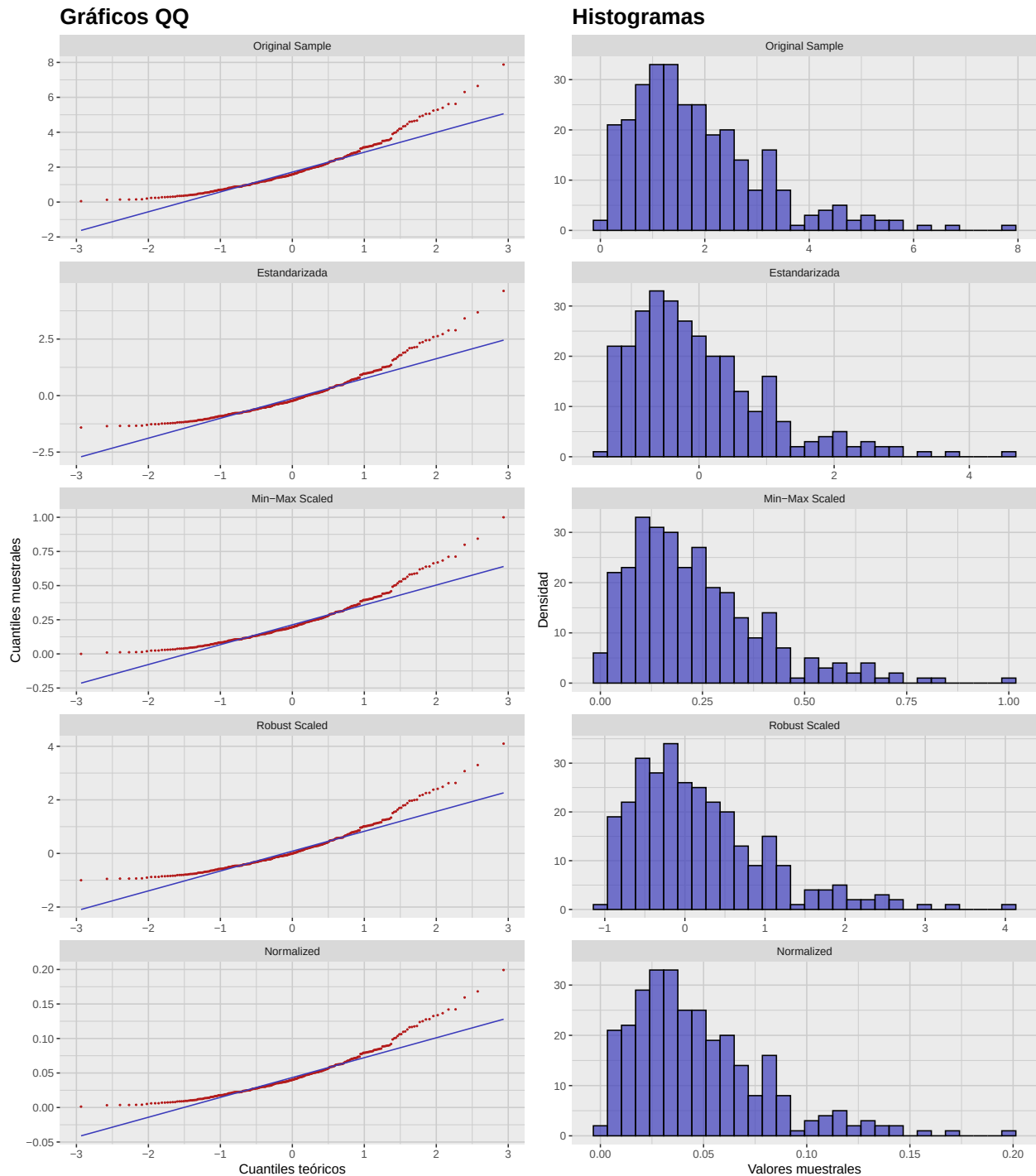


Figure 2.6: Comparación de distintos métodos de escalamiento aplicados a la muestra Gamma. Note que los QQ-plots son versiones reescaladas del gráfico QQ original y, por tanto, tienen la misma forma.

2.6 Actividad: Visualizando la no normalidad de muestras

Para cada una de las siguientes distribuciones, construya una muestra de tamaño 300 y calcule la asimetría y curtosis muestrales. Después, construya y compare sus gráficos QQ e histogramas.

1. Distribución exponencial con tasa $\lambda = 1$.
2. Distribución uniforme en $[0, 1]$.
3. Distribución beta con $\alpha = 2$ y $\beta = 5$.
4. Distribución de Cauchy con localización $x_0 = 0$ y escala $\gamma = 1$.
5. Distribución ji-cuadrada con $k = 3$ grados de libertad.

Chapter 3

Pruebas de normalidad

Aunque los gráficos QQ son una buena forma de evaluar visualmente la normalidad, pueden ser subjetivos e imprecisos. Por ello, es necesario contar con pruebas estadísticas más objetivas y precisas para determinar si los datos siguen una distribución normal. En este capítulo exploraremos pruebas de normalidad comunes y algunas transformaciones para acercar los datos a la normalidad.

3.1 Pruebas de normalidad

Existen varias pruebas estadísticas para evaluar si un conjunto sigue una normal. Algunas de las más usadas son:

1. *Shapiro-Wilk*: basada en la correlación entre los datos y puntajes normales. Es potente para muestras pequeñas (menores a 50) y muy usada; puede ser demasiado sensible en muestras grandes.
2. *Kolmogorov-Smirnov*: compara la función de distribución empírica con la acumulada de la normal. Es no paramétrica y aplicable a cualquier tamaño, pero menos potente que Shapiro-Wilk en muestras pequeñas; sensible a parámetros de escala y localización.
3. *Anderson-Darling*: similar a Kolmogorov-Smirnov, pero da mayor peso a las colas. Potente en muestras pequeñas y ampliamente utilizada.
4. *Jarque-Bera*: basada en la asimetría y curtosis de los datos. Potente en muestras grandes; en muestras pequeñas puede ser demasiado sensible a leves desvíos.

Aplicaremos estas pruebas a la muestra Gamma creada en el capítulo anterior:

```
normality_tests_lookup_tbl <- tibble(  
  code = c("shapiro", "ks", "ad", "jb"),  
  label = c(  
    "Shapiro-Wilk",  
    "Kolmogorov-Smirnov",  
    "Anderson-Darling",  
    "Jarque-Bera"  
  ),  
)
```

```
)  
  
normality_test_gamma_sample_tbl <- gamma_sample_tbl %>%  
  summarise(  
    shapiro = tidy_shapiro_test(data = ., value_col = sample)$p_value,  
    ks = tidy_ks_test(data = ., value_col = sample)$p_value,  
    ad = tidy_ad_test(data = ., value_col = sample)$p_value,  
    jb = tidy_jb_test(data = ., value_col = sample)$p_value  
  ) %>%  
  pivot_longer(  
    everything(),  
    names_to = "test",  
    values_to = "p_value"  
  ) %>%  
  convert_codes_to_factor(  
    code_col = test,  
    lookup_tbl = normality_tests_lookup_tbl,  
    lookup_code_col = code,  
    lookup_label_col = label,  
    new_factor_col_name = test_factor  
  ) %>%  
  select(test_factor, p_value)
```

Los resultados de las pruebas de normalidad son los siguientes:

```
normality_test_gamma_sample_tbl
```

```
# A tibble: 4 x 2  
  test_factor      p_value  
  <fct>          <dbl>  
1 Shapiro-Wilk    1.58e-12  
2 Kolmogorov-Smirnov 7.93e- 3  
3 Anderson-Darling 9.02e-16  
4 Jarque-Bera      0
```

Como era de esperarse (la Gamma no es normal), ninguna prueba resulta satisfactoria.

3.2 Transformaciones

Para acercar los datos a la normalidad, pueden aplicarse transformaciones. Algunas comunes son:

1. *Box-Cox transformation*: This is a family of power transformations that can be applied only to data that is always positive. It is given by the formula:

$$\tilde{X} = \begin{cases} \frac{X^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \log(X) & \text{if } \lambda = 0 \end{cases}$$

donde λ es un parámetro.

2. *Yeo-Johnson transformation*: This transformation is similar to the Box-Cox transformation but can handle negative values. It is useful for data that is positively or negatively skewed. It is given by the formula:

$$\tilde{X} = \begin{cases} \frac{((X+1)^\lambda - 1)}{\lambda} & \text{if } X \geq 0, \lambda \neq 0 \\ \frac{-((-X+1)^{2-\lambda} - 1)}{2-\lambda} & \text{if } X < 0, \lambda \neq 2 \\ \log(X+1) & \text{if } X \geq 0, \lambda = 0 \\ \log(-X+1) & \text{if } X < 0, \lambda = 2 \end{cases}$$

donde λ es un parámetro.

Aplicaremos la transformación de Box-Cox a la muestra Gamma con $\lambda = 0, 0.25, 0.5, 0.75$ y 1:

```
box_cox_lookup_tbl <- tibble(
  code = c(
    "box_cox_0",
    "box_cox_025",
    "box_cox_05",
    "box_cox_075",
    "box_cox_1"
  ),
  label = c(
    "lambda = 0",
    "lambda = 0.25",
    "lambda = 0.5",
    "lambda = 0.75",
    "lambda = 1"
  ),
)

box_cox_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    box_cox_0 = bcPower(sample, lambda = 0),
    box_cox_025 = bcPower(sample, lambda = 0.25),
    box_cox_05 = bcPower(sample, lambda = 0.5),
    box_cox_075 = bcPower(sample, lambda = 0.75),
    box_cox_1 = bcPower(sample, lambda = 1)
  ) %>%
  pivot_longer(
    starts_with("box_cox_"),
    names_to = "lambda",
    values_to = "transformed_sample"
  ) %>%
  convert_codes_to_factor(
    code_col = lambda,
    lookup_tbl = box_cox_lookup_tbl,
    lookup_code_col = code,
```

```
lookup_label_col = label,  
new_factor_col_name = lambda_factor  
)  
  
box_cox_gamma_qq_plot <- box_cox_gamma_sample_tbl %>%  
  ggplot(aes(sample = transformed_sample)) +  
  facet_wrap(  
    ~lambda_factor,  
    nrow = 5,  
    scales = "free"  
  ) +  
  geom_qq(  
    color = c_palette["C red"],  
    size = 0.3  
  ) +  
  geom_qq_line(  
    color = c_palette["C blue"],  
    linewidth = 0.5  
  ) +  
  labs(  
    title = "Gráficos QQ",  
    x = "Cuantiles teóricos",  
    y = "Cuantiles muestrales"  
  ) +  
  theme_krul()  
  
box_cox_gamma_histogram <- box_cox_gamma_sample_tbl %>%  
  ggplot(aes(x = transformed_sample)) +  
  facet_wrap(  
    ~lambda_factor,  
    nrow = 5,  
    scales = "free"  
  ) +  
  geom_histogram(  
    bins = 30,  
    fill = c_palette["C blue"],  
    color = "black",  
    alpha = 0.7  
  ) +  
  labs(  
    title = "Histogramas",  
    x = "Valores muestrales",  
    y = "Densidad"  
  ) +
```

```
theme_krul()

box_cox_gamma_plot <- box_cox_gamma_qq_plot + box_cox_gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Comparación de transformaciones Box-Cox con distintos valores de lambda",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

box_cox_gamma_plot
```

Comparación de transformaciones Box-Cox con distintos valores de λ

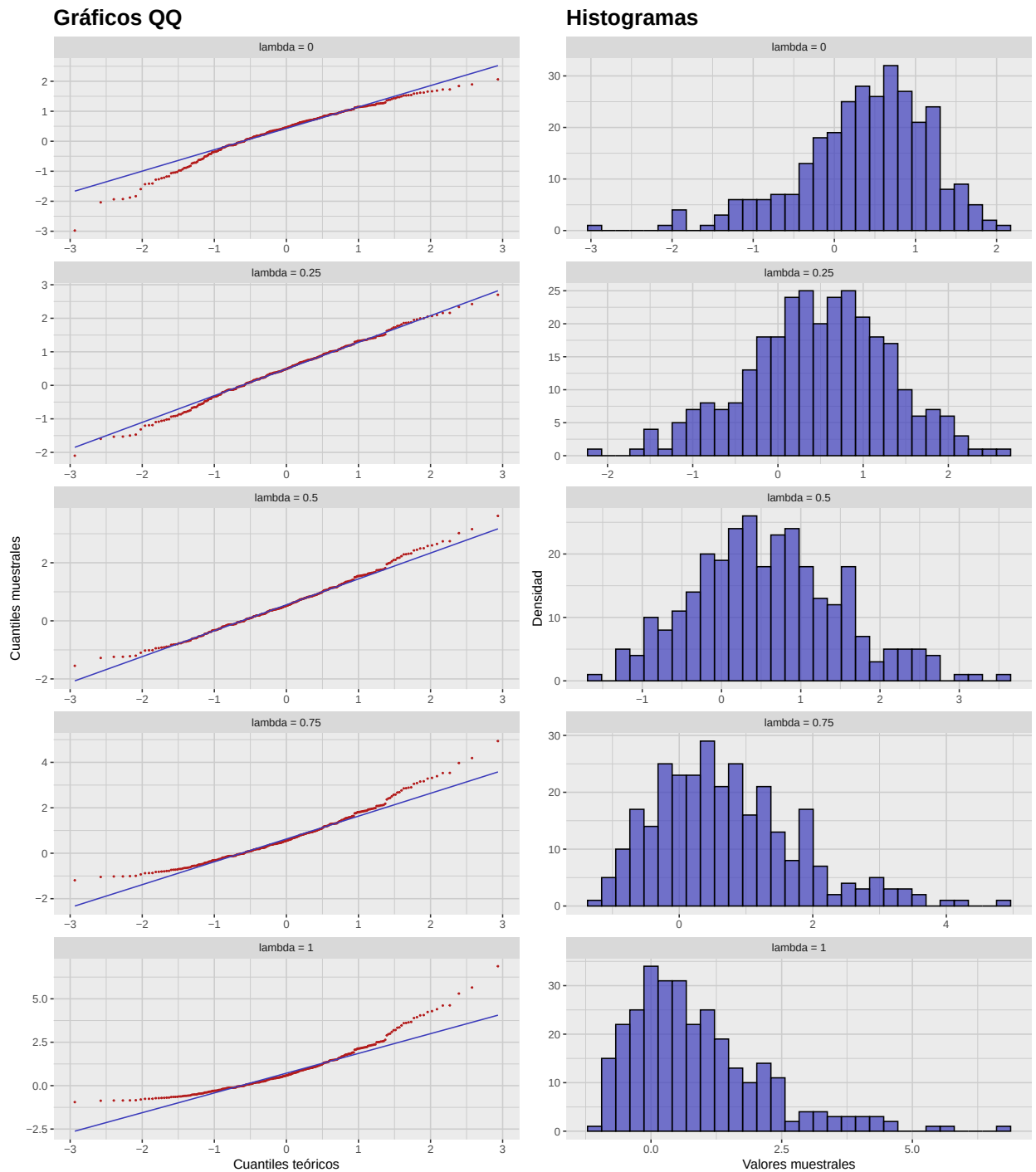


Figure 3.1: Comparación de distintas transformaciones de Box-Cox aplicadas a la muestra Gamma, con parámetro $\lambda = 0, 0.25, 0.5, 0.75$ y 1.

3.3 Optimización de normalidad (Box–Cox)

Como vimos, la transformación de Box–Cox puede acercar los datos a la normalidad. La elección de λ es crucial: buscamos el valor que maximiza la log-verosimilitud. Esto puede realizarse con `car::powerTransform`, que estima el λ óptimo. La figura muestra el perfil de log-verosimilitud para la muestra Gamma:

```
# Intercept-only model
fit <- lm(sample ~ 1, data = gamma_sample_tbl)

# Estimate lambda
bc <- powerTransform(fit)

# Extract log-likelihood profile
bc_prof <- boxCox(fit, lambda = seq(-2, 2, 0.1), plotit = FALSE)

# Convert to tibble for ggplot
df <- tibble(lambda = bc_prof$x, loglik = bc_prof$y)

ggplot(df, aes(x = lambda, y = loglik)) +
  geom_line(color = c_pal("C blue"), linewidth = 1) +
  geom_vline(xintercept = bc$lambda, linetype = "dashed", color = c_pal("C red")) +
  labs(
    title = "Transformación de Box-Cox",
    x = expression(lambda),
    y = "Log-verosimilitud"
  ) +
  theme_krul()
```

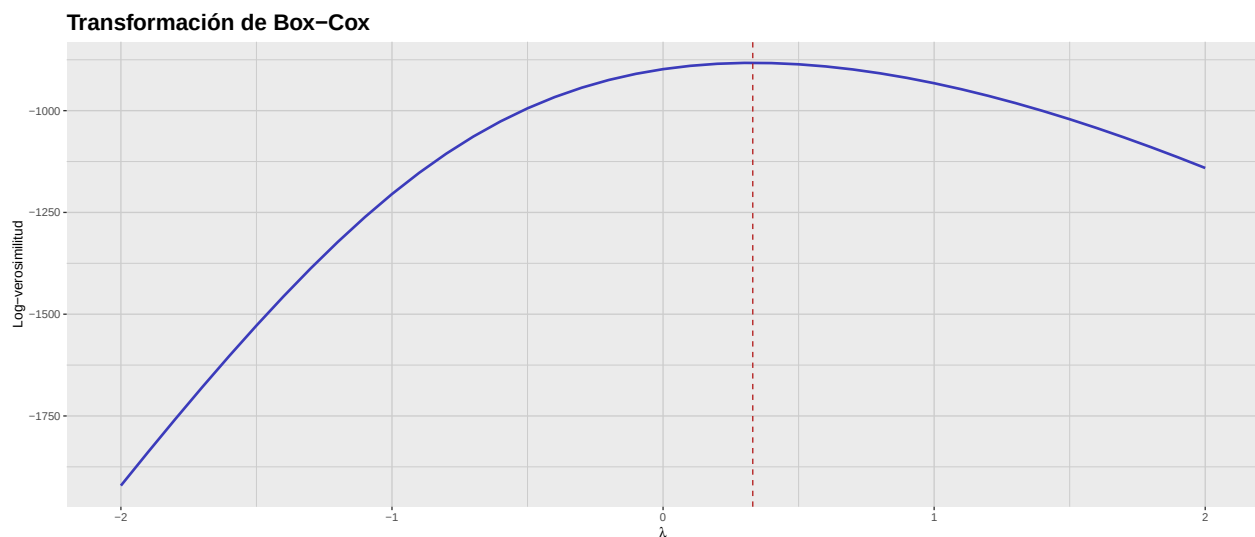


Figure 3.2: Perfil de log-verosimilitud para la transformación de Box–Cox aplicada a la muestra Gamma. La línea discontinua indica el valor óptimo de λ .

Finalmente, extraemos el λ óptimo y transformamos los datos con ese valor:

```
optimal_lambda <- bc$lambda

optimal_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    transformed_sample = bcPower(sample, lambda = optimal_lambda)
  )

optimal_gamma_sample_tbl %>%
  summarise(
    shapiro = tidy_shapiro_test(data = ., value_col = transformed_sample)$p_value,
    ks = tidy_ks_test(data = ., value_col = transformed_sample)$p_value,
    ad = tidy_ad_test(data = ., value_col = transformed_sample)$p_value,
    jb = tidy_jb_test(data = ., value_col = transformed_sample)$p_value
  ) %>%
  pivot_longer(
    everything(),
    names_to = "test",
    values_to = "p_value"
  ) %>%
  convert_codes_to_factor(
    code_col = test,
    lookup_tbl = normality_tests_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = test_factor
  ) %>%
  select(test_factor, p_value)
```

```
# A tibble: 4 x 2
  test_factor      p_value
  <fct>           <dbl>
1 Shapiro-Wilk    0.974
2 Kolmogorov-Smirnov 1.000
3 Anderson-Darling 0.977
4 Jarque-Bera     0.878
```

Como se observa, la transformación de Box-Cox con el valor óptimo de λ mejora significativamente la normalidad de los datos, pues todas las pruebas arrojan valores p altos. A continuación se presentan el gráfico QQ y el histograma de los datos transformados:

```
optimal_gamma_qq_plot <- optimal_gamma_sample_tbl %>%
  ggplot(aes(sample = transformed_sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
```

```
) +  
geom_qq_line(  
  color = c_palette["C blue"],  
  linewidth = 0.5  
) +  
labs(  
  title = "Gráfico QQ",  
  x = "Cuantiles teóricos",  
  y = "Cuantiles muestrales"  
) +  
theme_krul()  
  
optimal_gamma_histogram <- optimal_gamma_sample_tbl %>%  
  ggplot(aes(x = transformed_sample)) +  
  geom_histogram(  
    bins = 30,  
    fill = c_palette["C blue"],  
    color = "black",  
    alpha = 0.7  
) +  
  labs(  
    title = "Histograma",  
    x = "Valores muestrales",  
    y = "Densidad"  
) +  
  theme_krul()  
  
optimal_gamma_plot <- optimal_gamma_qq_plot + optimal_gamma_histogram +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Transformación Gamma óptima",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
  
optimal_gamma_plot
```

Transformación Gamma óptima

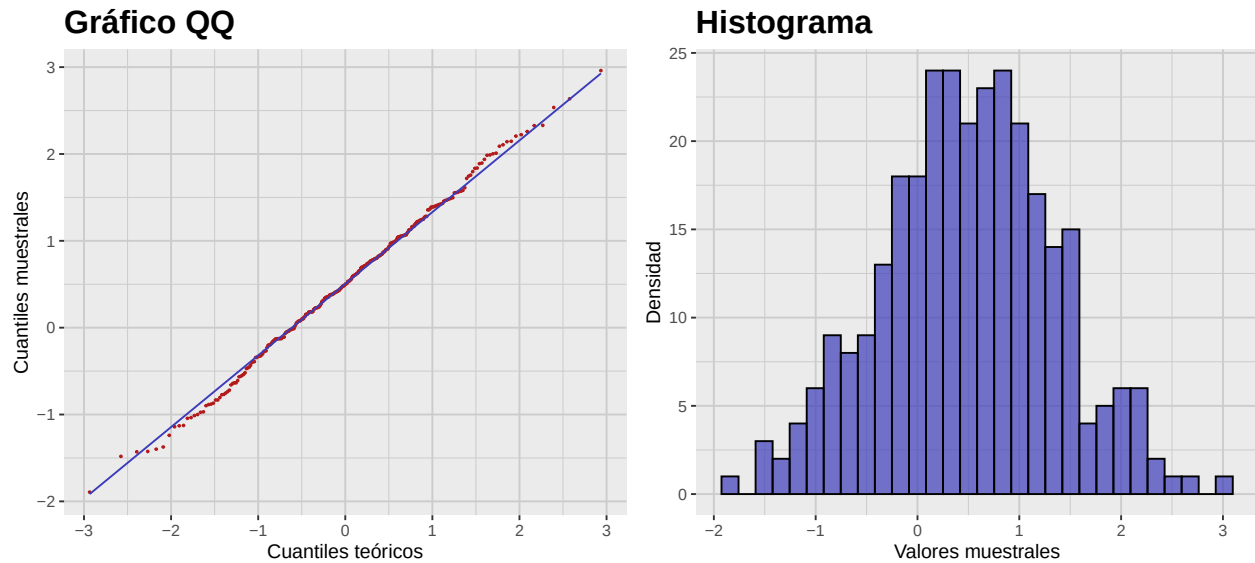


Figure 3.3: Gráfico QQ e histograma de la transformación Gamma óptima. Nótese el mejor ajuste a la normalidad en comparación con la muestra Gamma original.

Part II

Análisis de Componentes Principales

Chapter 4

Introducción al análisis multivariante

En este capítulo introducimos conceptos básicos del análisis multivariante, enfocándonos en el vector de medias y la matriz de covarianzas de un vector aleatorio. Estos conceptos son fundamentales para entender relaciones entre múltiples variables en un conjunto de datos.

4.1 El vector de medias y la matriz de covarianzas

Definición 4.1.1. Let Π be a population and let

$$X : \Pi \rightarrow \mathbb{R}^p$$

be a random (row) vector. The *mean vector* of X is defined as

$$\mu = \mathbb{E}[X] = [\mathbb{E}[X_1], \mathbb{E}[X_2], \dots, \mathbb{E}[X_p]] \quad (4.1)$$

where X_i is the i -th component of the random vector X . The *covariance matrix* of X is defined as

$$\Sigma = \text{Cov}[X] = \mathbb{E}[(X - \mu)^T(X - \mu)] = [\text{Cov}[X_i, X_j]]_{i,j=1}^p \quad (4.2)$$

where $\text{Cov}[X_i, X_j] = \mathbb{E}[(X_i - \mathbb{E}[X_i])(X_j - \mathbb{E}[X_j])]$ is the covariance between the i -th and j -th components of the random vector X .

Comentario 4.1.2. Recuerde que $\text{Cov}[X_i, X_j] = \text{Cov}[X_j, X_i]$, por lo que la matriz es simétrica. Además, $\text{Cov}[X_i, X_i] = \text{Var}[X_i]$, la varianza de la i -ésima componente de X . Por ello también se le llama *matriz varianza-covarianza*.

Ahora, dada una muestra de tamaño n de Π , su *matriz muestral* es de tamaño $n \times p$ y cada renglón contiene una observación del vector X . En este contexto, el vector de medias muestral y la matriz de covarianzas muestral se definen así:

Definición 4.1.3. Given a sample of size n from the population Π , with associated sample matrix x , we define its associated *sample mean vector* as

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \left[\frac{1}{n} \sum_{i=1}^n x_{i,1}, \frac{1}{n} \sum_{i=1}^n x_{i,2}, \dots, \frac{1}{n} \sum_{i=1}^n x_{i,p} \right] \quad (4.3)$$

where x_i is the i -th row of the sample matrix x and $x_{i,j}$ is the j -th component of the i -th row. The *sample covariance matrix* is defined as

$$\hat{\Sigma} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^T (x_i - \bar{x}) \quad (4.4)$$

where $\hat{\Sigma}$ is a $p \times p$ matrix and $\hat{\Sigma}_{i,j} = \frac{1}{n-1} \sum_{k=1}^n (x_{k,i} - \bar{x}_i)(x_{k,j} - \bar{x}_j)$ is the sample covariance between the i -th and j -th components of the random vector X .

Comentario 4.1.4. La matriz de covarianzas muestral es un estimador insesgado de la matriz de covarianzas de X .

Ejemplo 4.1.5. Considere la siguiente matriz muestral de tamaño $n = 5$ de una población Π :

$$\begin{bmatrix} 2 & 5 & 7 \\ 4 & 3 & 6 \\ 6 & 4 & 5 \\ 8 & 2 & 4 \\ 10 & 1 & 3 \end{bmatrix}$$

Para obtener el vector de medias muestral, calcule:

$$\begin{aligned} \bar{x}_1 &= \frac{2 + 4 + 6 + 8 + 10}{5} = 6 \\ \bar{x}_2 &= \frac{5 + 3 + 4 + 2 + 1}{5} = 3 \\ \bar{x}_3 &= \frac{7 + 6 + 5 + 4 + 3}{5} = 5 \end{aligned}$$

In other words,

$$\bar{x} = [6, 3, 5]$$

La varianza muestral de X_j es:

$$\hat{\Sigma}_{X_j} = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2$$

Por lo tanto,

$$\begin{aligned} \hat{\Sigma}_{X_1} &= \frac{(2-6)^2 + (4-6)^2 + (6-6)^2 + (8-6)^2 + (10-6)^2}{4} = 10 \\ \hat{\Sigma}_{X_2} &= \frac{(5-3)^2 + (3-3)^2 + (4-3)^2 + (2-3)^2 + (1-3)^2}{4} = 2.5 \\ \hat{\Sigma}_{X_3} &= \frac{(7-5)^2 + (6-5)^2 + (5-5)^2 + (4-5)^2 + (3-5)^2}{4} = 2.5 \end{aligned}$$

La covarianza muestral entre X_j y X_k es:

$$\hat{\Sigma}_{X_j, X_k} = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

Calculating the sample covariances, we have:

$$\begin{aligned}\hat{\Sigma}_{X_1, X_2} &= \frac{(2-6)(5-3) + (4-6)(3-3) + (6-6)(4-3) + (8-6)(2-3) + (10-6)(1-3)}{4} \\ &= \frac{(-8) + 0 + 0 + (-2) + (-8)}{4} = -4.5 \\ \hat{\Sigma}_{X_1, X_3} &= \frac{(2-6)(7-5) + (4-6)(6-5) + (6-6)(5-5) + (8-6)(4-5) + (10-6)(3-5)}{4} \\ &= \frac{(-8) + (-2) + 0 + (-2) + (-8)}{4} = -5 \\ \hat{\Sigma}_{X_2, X_3} &= \frac{(5-3)(7-5) + (3-3)(6-5) + (4-3)(5-5) + (2-3)(4-5) + (1-3)(3-5)}{4} \\ &= \frac{4 + 0 + 0 + 1 + 4}{4} = 2.25\end{aligned}$$

La matriz de covarianzas resultante es:

$$\hat{\Sigma} = \begin{bmatrix} 10 & -4.5 & -5 \\ -4.5 & 2.5 & 2.25 \\ -5 & 2.25 & 2.5 \end{bmatrix}$$

4.2 La distancia de Mahalanobis

Definición 4.2.1. Assume that we have a population Π with a random vector X that has mean vector μ and covariance matrix Σ . If Σ is invertible, then the *Mahalanobis distance* between two points x and y in $\mathbb{R}^p$ is defined as:

$$d_M(x, y) = \sqrt{(x - y)\Sigma^{-1}(x - y)^T} \quad (4.5)$$

In the context of sample data, the Mahalanobis distance can be computed using the sample covariance matrix $\hat{\Sigma}$:

Definición 4.2.2. Given a sample of size n from the population Π , with associated sample matrix x and sample covariance matrix $\hat{\Sigma}$, the *sample Mahalanobis distance* between two points x_i and x_j is defined as:

$$d_M(x_i, x_j) = \sqrt{(x_i - x_j)\hat{\Sigma}^{-1}(x_i - x_j)^T} \quad (4.6)$$

Comentario 4.2.3. The Mahalanobis distance is a measure of the distance between two points in a multivariate space, taking into account the correlations between the variables. It is particularly useful for identifying outliers in multivariate data.

Para ilustrar, consideremos el siguiente conjunto de datos. Iniciamos con una muestra aleatoria de tamaño $n = 300$ de una uniforme en $[0, 10]$.

```
set.seed(123)
n <- 300
x_data <- runif(n, min = 0, max = 10)
```

Now we will construct some error vector ϵ from a normal distribution with mean 0 and standard deviation 1:

```
epsilon_data <- rnorm(n, mean = 0, sd = 1)
```

We can now construct the vector

```
y_data <- x_data + epsilon_data
```

and put it together to create a tibble:

```
sample_data_tbl <- tibble(  
  x = x_data,  
  y = y_data  
)
```

The associated scatter plot of the data is shown below:

```
sample_data_plot <- sample_data_tbl %>%  
  ggplot(aes(x = x_data, y = y_data)) +  
  geom_point(  
    color = "black",  
    size = 0.2,  
    alpha = 1  
  ) +  
  labs(  
    title = "Datos de muestra",  
    x = "X",  
    y = "Y"  
  ) +  
  coord_fixed() +  
  theme_krul()  
  
sample_data_plot
```

Datos de muestra

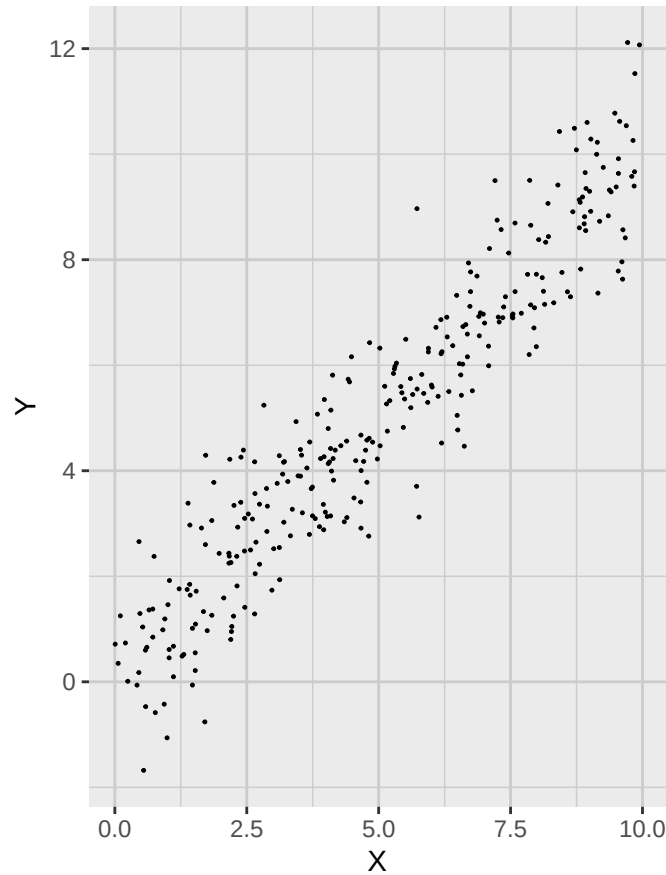


Figure 4.1: Diagrama de dispersión de los datos de muestra. Nótese la alta correlación entre las variables X e Y.

We now add two extra points to this plot:

```
extra_points_tbl <- tibble(  
  x_extra = c(1, 5),  
  y_extra = c(1, 2)  
)  
  
extra_sample_data_plot <- sample_data_plot +  
  geom_point(  
    data = extra_points_tbl,  
    aes(x = x_extra, y = y_extra),  
    color = c_pal("C red"),  
    size = 1.5  
  )  
  
extra_sample_data_plot
```

Datos de muestra

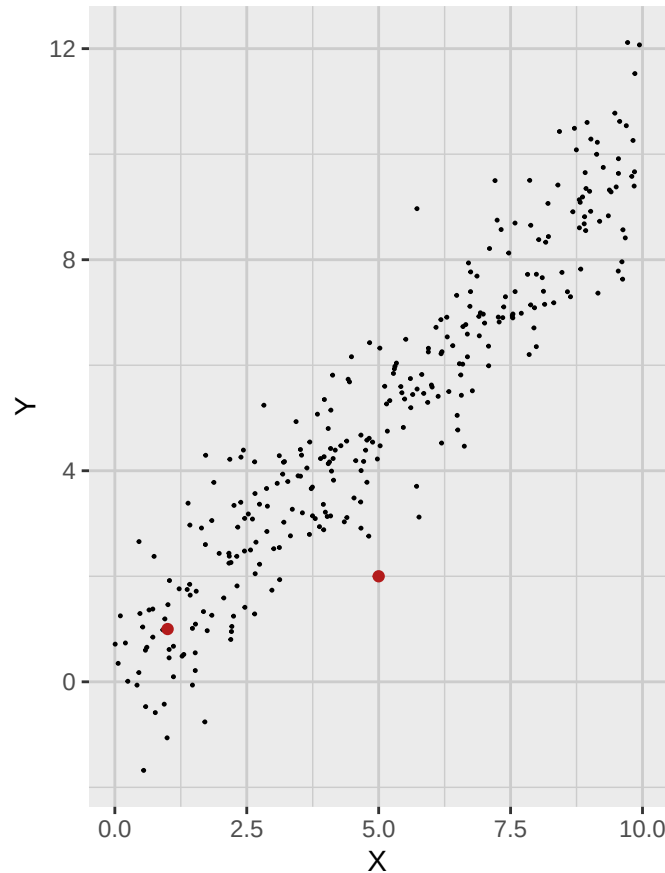


Figure 4.2: Dispersión con puntos adicionales. Nótese que uno de ellos es claramente atípico.

Notice that one of these points is clearly an outlier. We would like to have some concrete measure to show this, so we start by computing the sample mean vector which represents the center of our data. For future reference, we will also compute the sample covariance matrix.

```
sample_mean_vector <- colMeans(sample_data_tbl)
sample_covariance_matrix <- cov(sample_data_tbl)

sample_mean_tbl <- tibble(
  x_bar = sample_mean_vector[1],
  y_bar = sample_mean_vector[2]
)
```

But now if we use the Euclidean distance, we obtain that it is actually the outlier point that is closer to the sample mean vector:

```
euclidean_sample_data_plot <- extra_sample_data_plot +
  geom_point(
    data = sample_mean_tbl,
```

```

aes(x = x_bar, y = y_bar),
color = c_pal("C red"),
size = 1.5
) +
geom_circle(
aes(x0 = sample_mean_vector[1], y0 = sample_mean_vector[2], r = 4),
color = c_pal("C blue"),
linewidth = 0.6,
fill = NA
)
euclidean_sample_data_plot

```

Datos de muestra

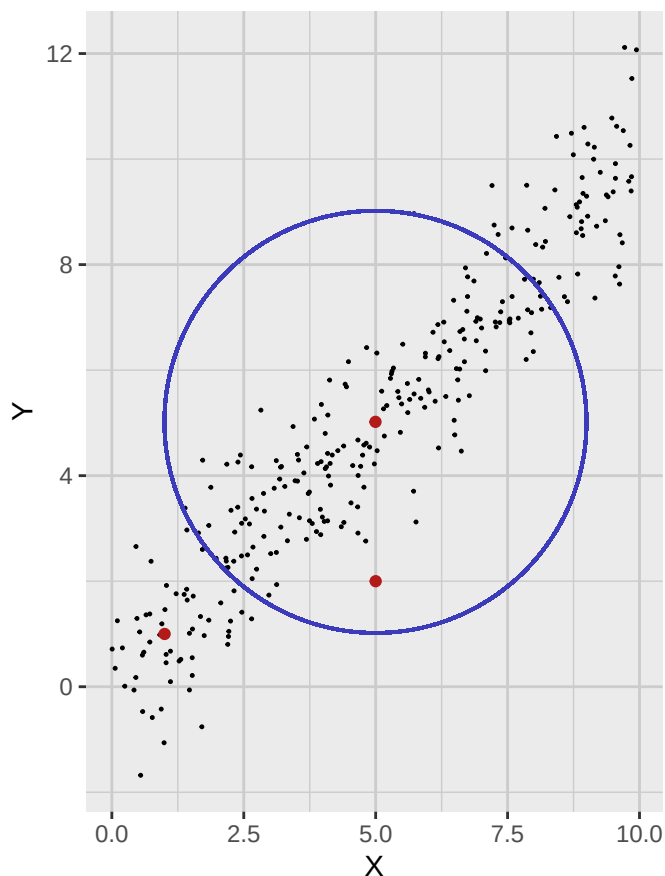


Figure 4.3: Dispersión con vector de media muestral y un círculo de distancia euclidiana constante al centro de los datos.

On the other hand, if we compute the Mahalanobis distance, we obtain that the outlier point is actually further away from the sample mean vector:

```
annotated_sample_data_plot <- extra_sample_data_plot +
  geom_point(
    data = sample_mean_tbl,
    aes(x = x_bar, y = y_bar),
    color = c_pal("C red"),
    size = 1.5
  ) +
  stat_ellipse(
    type = "norm",
    level = 0.8647,
    color = c_pal("C blue"),
    linewidth = 0.6
  )
)
```

annotated_sample_data_plot

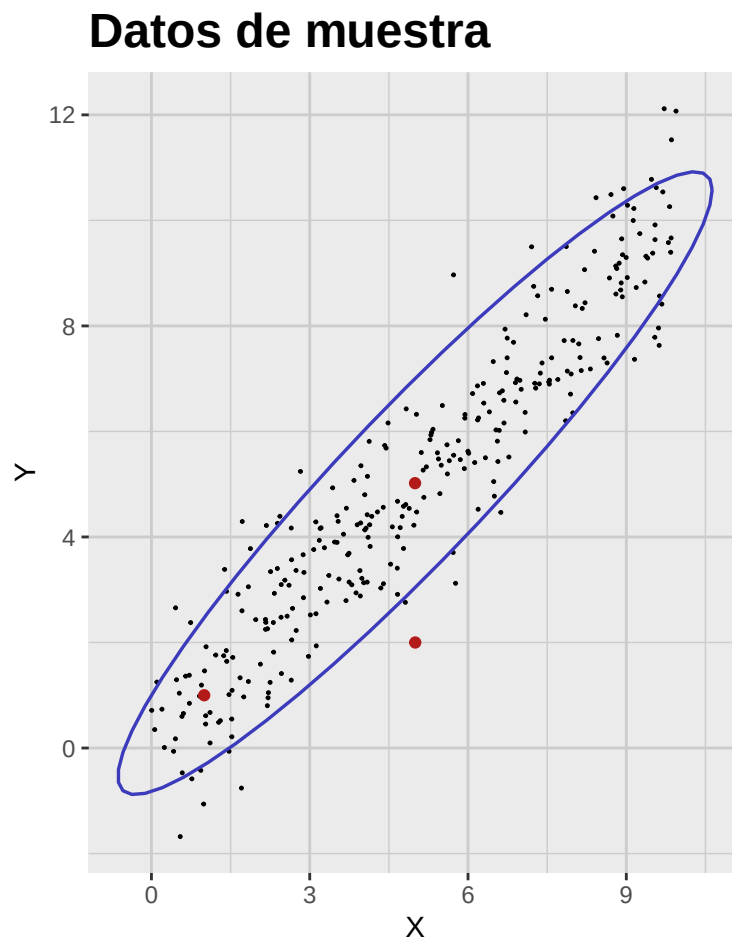


Figure 4.4: Dispersión con vector de media muestral y una elipse de distancia de Mahalanobis constante al centro de los datos.

4.3 Escalamiento y estandarización

Suponga ahora que tenemos un vector aleatorio X con media μ y covarianzas Σ . Definimos un nuevo vector Y como:

$$Y^T = AX^T + b$$

donde A es una matriz $p \times p$ y b un vector de dimensión p . El vector de medias y la matriz de covarianzas de Y se obtienen como:

$$\mu_Y^T = \mathbb{E}[Y]^T = A\mu^T + b$$

and

$$\Sigma_Y = \text{Cov}[Y] = A\Sigma A^T$$

Más adelante mostraremos que siempre es posible encontrar A y b tales que $\mu_Y = 0$ y $\Sigma_Y = I_p$ (identidad $p \times p$). A este proceso se le conoce como *estandarización* de X .

4.4 Actividad: La geometría de los datos multivariantes

Parte A: Ejercicios teóricos

1. Demuestre que la distancia de Mahalanobis se reduce a la euclídea si la covarianza es la identidad.
2. Muestre que la distancia de Mahalanobis es invariante bajo transformaciones afines.
3. Muestre que al estandarizar cada variable, la matriz de covarianzas pasa a ser la matriz de correlaciones.
4. Si X_1 y X_2 son normales estándar independientes, muestre que $d_M(X, 0)^2$ sigue una ji-cuadrada con 2 g.l., donde d_M es la distancia al origen.

Parte B: Ejercicios prácticos

1. Genere un conjunto 2D de 50 puntos (dos variables correlacionadas).
2. Calcule el vector de medias muestral y la matriz de covarianzas.
3. Escriba una función en R que calcule la distancia de Mahalanobis de cada punto a la media.
4. Identifique el punto con mayor distancia de Mahalanobis.
5. Grafique el conjunto con `ggplot2`.
6. Dibuje elipses de distancia de Mahalanobis 1, 2 y 3.
7. Coloree puntos según si su distancia supera 1, 2 o 3.
8. Cree un conjunto de 3 variables.
9. Calcule distancias de Mahalanobis en 3D.
10. Grafique pares de variables con elipses proyectadas.
11. Compare distancias euclídeas a la media con distancias de Mahalanobis.

Parte C: Ejercicios avanzados

1. Simulate 5 points in 3 dimensions with one variable as a linear combination of the others.
2. Try to compute Mahalanobis distance and discuss singularity of covariance.
3. Apply a linear transformation to a 2D dataset and verify invariance of Mahalanobis distances.
4. Write an R function that returns points forming a Mahalanobis-distance ellipse.
5. Overlay Mahalanobis-distance ellipses of arbitrary radius on a scatter plot.

Chapter 5

Distribución normal multivariante

5.1 Definición y propiedades básicas

Definición 5.1.1. Sea μ un vector (renglón) y Σ una matriz simétrica definida positiva. Decimos que un vector aleatorio (renglón) X tiene *distribución normal multivariante* con media μ y matriz de covarianzas Σ , en símbolos $X \sim \mathcal{N}_p(\mu, \Sigma)$, si su densidad es

$$\frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(X - \mu)\Sigma^{-1}(X - \mu)^T\right) dX$$

Comentario 5.1.2. Algunas observaciones sobre la definición:

1. Al igual que en el caso univariado, la normal multivariante queda determinada por μ y Σ .
2. Σ debe ser simétrica y definida positiva, garantizando que la distribución esté bien definida y el exponente sea negativo definido.
3. El término

$$|\Sigma| = \det(\Sigma)$$

es la *varianza generalizada*. Puede interpretarse como una medida de la “dispersión” en el espacio de dimensión p .

4. El término $(X - \mu)\Sigma^{-1}(X - \mu)^T$ es el cuadrado de la distancia de Mahalanobis entre X y la media μ .

Como en el caso univariado, la normal multivariante se destaca por el Teorema Central del Límite (TCL), que en términos generales afirma que la suma de muchas variables i.i.d. (debidamente estandarizadas) converge a una normal. El enunciado preciso es:

Teorema 5.1.3 (Teorema central del límite multivariante). *Let $\{X_i\}_{i=1}^{\infty}$ be a sequence of independent and identically distributed random vectors in $\mathbb{R}^p$ with mean vector μ and covariance matrix Σ . Define the sample mean*

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i.$$

Then, as $n \rightarrow \infty$,

$$\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}_p(0, \Sigma),$$

donde $\xrightarrow{d}$ denota convergencia en distribución y $\mathcal{N}_p(0, \Sigma)$ es la normal multivariante p -dimensional con media 0 y covarianzas Σ .

Comentario 5.1.4. Aun cuando el TCL destaca a la normal multivariante, no es la única relevante en estadística multivariante. De hecho, muchos datos reales no siguen normalidad, y existen métodos para manejarla; además, técnicas como regresión lineal, PCA y clustering no arrojan buenos resultados aplicadas a una normal multivariante.

5.2 Contornos de la normal multivariante

Observe que la densidad de la normal multivariante es gaussiana en p dimensiones. Por ello, sus contornos son elipsoides centrados en μ y su forma está controlada por Σ . La probabilidad de estos contornos está gobernada por la χ^2 así: sea $0 \leq \varpi \leq 1$ y c tal que

$$\mathbb{P}(\chi_p^2 \leq c) = \varpi$$

Entonces, el contorno que contiene proporción ϖ de masa de probabilidad es el conjunto de X que satisfacen

$$(X - \mu)\Sigma^{-1}(X - \mu)^T = c$$

En la figura siguiente se muestran contornos de normales bivariantes con distintas covarianzas. Son elipses centradas en $\mu = (0, 0)$; el gradiente de color representa la densidad (más oscuro, mayor densidad).

```
# Add seed for reproducibility
set.seed(123)

# Parameters
mu <- c(0, 0)
Sigma_1 <- matrix(c(1, 0, 0, 1), nrow = 2)
Sigma_2 <- matrix(c(0.3, 0, 0, 1.5), nrow = 2)
Sigma_3 <- matrix(c(1, 0.6, 0.6, 1), nrow = 2)
Sigma_4 <- matrix(c(1, -0.6, -0.6, 1), nrow = 2)

# Function to generate contour + scatter plot
plot_bivariate_normal <- function(mu, Sigma, n_samples = 1000,
                                   xlim = c(-3, 3), ylim = c(-3, 3),
                                   grid_length = 100) {

  # Create grid
  x <- seq(xlim[1], xlim[2], length.out = grid_length)
  y <- seq(ylim[1], ylim[2], length.out = grid_length)
  grid <- expand.grid(x = x, y = y)
  grid$z <- dmvnorm(grid, mean = mu, sigma = Sigma)
```

```
# Sample points
samples <- rmvnorm(n_samples, mean = mu, sigma = Sigma)
colnames(samples) <- c("x", "y")
samples <- as_tibble(samples)

# Plot
normal_plot <- ggplot() +
  geom_contour(data = grid, aes(x, y, z = z, color = after_stat(level))) +
  geom_point(
    data = samples,
    aes(x, y),
    color = c_pal("C red"),
    alpha = 0.5,
    size = 0.1
  ) +
  scale_color_viridis_c(option = "mako") +
  coord_cartesian(xlim = xlim, ylim = ylim) +
  labs(
    title = "",
    x = expression(X[1]), y = expression(X[2]),
    color = "Densidad"
  ) +
  theme_krul()

return(normal_plot)
}

normal_plot_1 <- plot_bivariate_normal(mu, Sigma_1)
normal_plot_2 <- plot_bivariate_normal(mu, Sigma_2)
normal_plot_3 <- plot_bivariate_normal(mu, Sigma_3)
normal_plot_4 <- plot_bivariate_normal(mu, Sigma_4)

normal_plot <- (normal_plot_1 + normal_plot_2) /
  (normal_plot_3 + normal_plot_4) +
  plot_layout(widths = c(1, 1), heights = c(1, 1)) +
  plot_annotation(
    title = "Contornos de la normal bivalente ",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
)
```

normal_plot

Contornos de la normal bivalente

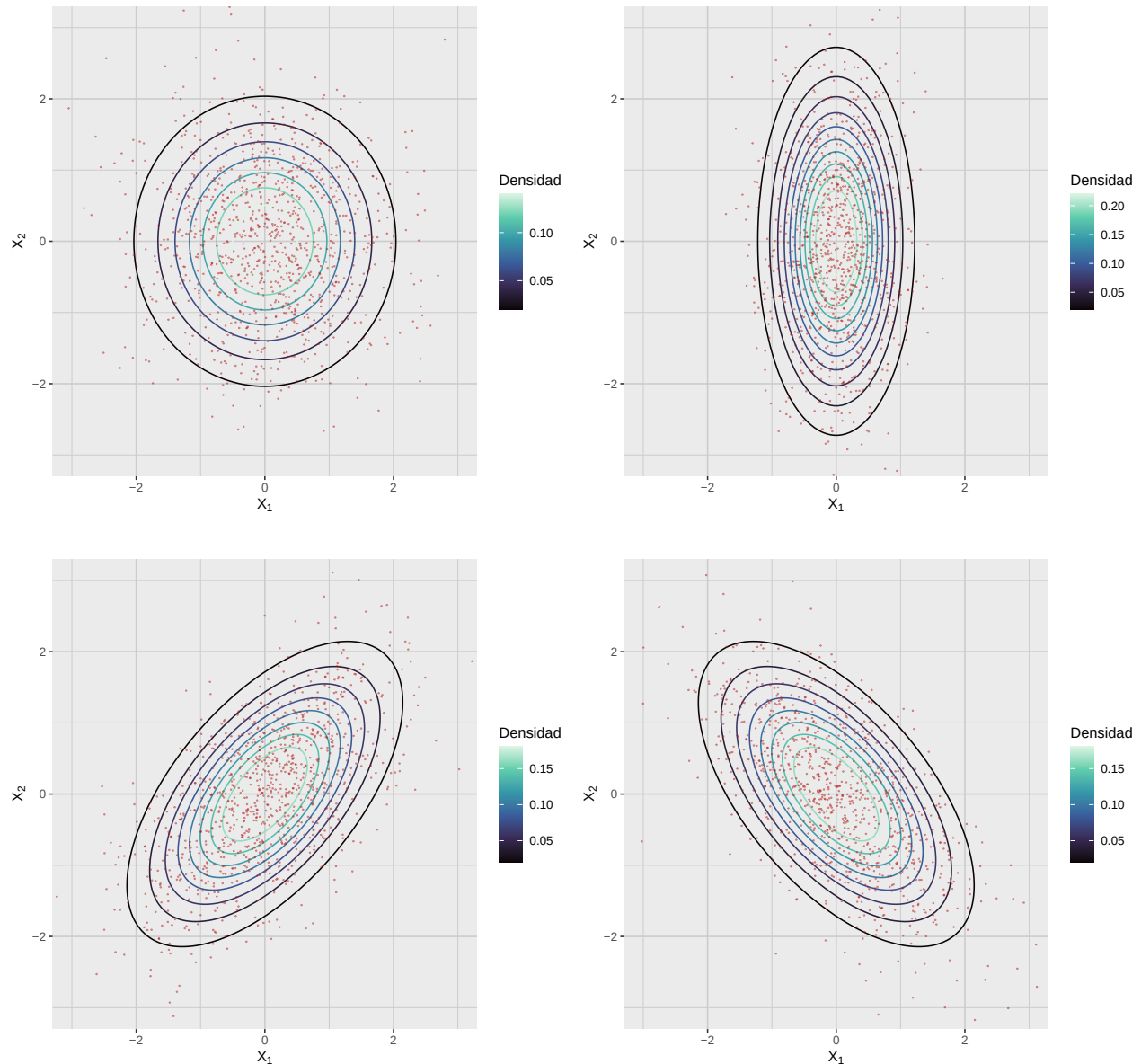


Figure 5.1: Distribución normal bivalente con distintas matrices de covarianzas

5.3 Estandarización de la normal multivariante

Al igual que en el caso univariado, si $X \sim \mathcal{N}_p(\mu, \Sigma)$ podemos estandarizar para obtener $Z \sim \mathcal{N}_p(0, I_p)$, donde I_p es la identidad de tamaño p . Para describirlo, recordemos algunos resultados de álgebra

lineal.

Teorema 5.3.1 (Singular Value Decomposition). *Let Σ be a symmetric positive definite matrix of size $p \times p$. Then, there exists an orthogonal matrix Q and a diagonal matrix Λ with positive entries such that*

$$\Sigma = Q\Lambda Q^T$$

Nota 5.3.2. Recordemos que una matriz Q es ortogonal si $Q^T Q = I_p$; sus columnas son ortonormales y forman una base de $\mathbb{R}^p$.

Comentario 5.3.3. La descomposición en valores singulares (SVD) permite descomponer una matriz en valores (diagonal de Λ , también *autovalores*) y vectores singulares (columnas de Q , *autovectores*). Es útil para entender la geometría de la normal multivariante.

Notice that, since Σ is positive definite, the diagonal entries of

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_p \end{bmatrix}$$

are all positive. In particular, we can define a new matrix

$$D = \begin{bmatrix} \sqrt{\lambda_1} & 0 & \cdots & 0 \\ 0 & \sqrt{\lambda_2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sqrt{\lambda_p} \end{bmatrix}$$

satisfying $D^2 = \Lambda$. If we now take

$$\sigma = DQ^T$$

then

$$\Sigma = \sigma^T \sigma$$

In particular, we can define the *standardization* of the random vector X as

$$Z = (X - \mu)\sigma^{-1} = (X - \mu)QD^{-1} = (X - \mu)Q\Lambda^{-1/2}$$

in which case $Z \sim \mathcal{N}_p(0, I_p)$.

5.4 Geometric Interpretation of the SVD

Earlier in this chapter we considered the Normal Multivariate Distribution with mean

$$\mu = [0, 0]$$

and covariance matrix

$$\Sigma = \begin{bmatrix} 1 & 0.6 \\ 0.6 & 1 \end{bmatrix}$$

According to the SVD, we can write this covariance matrix as

$$\Sigma = Q\Lambda Q^T$$

for some orthogonal matrix Q and diagonal matrix Λ . In this case, we can compute the SVD of Σ as follows:

```
# Example mean and covariance
mu <- c(0, 0)
Sigma <- matrix(c(
  1, 0.6,
  0.6, 1
), 2, 2)

# Compute eigenvectors and eigenvalues
eig <- eigen(Sigma)
vectors <- eig$vectors
values <- eig$values
```

We obtain:

$$Q = \begin{bmatrix} 0.7071068 & -0.7071068 \\ 0.7071068 & 0.7071068 \end{bmatrix}$$

and

$$\Lambda = \begin{bmatrix} 1.6 & 0 \\ 0 & 0.4 \end{bmatrix}$$

where the singular vectors are the columns of Q and the singular values are the diagonal entries of Λ . Geometrically, the singular vectors represent the directions of the axes of the ellipsoid that defines the contours of the normal multivariate distribution, and the square root of the singular values represent the lengths of these axes.

```
# Scale eigenvectors by sqrt(eigenvalues) for plotting
vec_data <- tibble(
  x0 = mu[1],
  y0 = mu[2],
  x1 = mu[1] + sqrt(values) * vectors[1, ],
  y1 = mu[2] + sqrt(values) * vectors[2, ]
)

# Generate points of the 1-Mahalanobis-distance ellipse
ellipse_points <- as_tibble(ellipse(Sigma, centre = mu, level = 0.393))
# level = 0.393 corresponds to ~1 standard deviation for 2D

# Plot
ggplot() +
  geom_path(
```

```
data = ellipse_points,  
aes(x = x, y = y),  
color = c_pal("C blue"),  
linewidth = 1  
)+  
geom_segment(  
data = vec_data,  
aes(x = x0, y = y0, xend = x1, yend = y1),  
arrow = arrow(length = unit(0.2, "cm")),  
color = c_pal("C green"),  
linewidth = 1  
)+  
geom_point(  
aes(x = mu[1], y = mu[2]),  
color = c_pal("C red"),  
size = 3  
)+  
coord_equal() +  
labs(  
title = "Interpretación geométrica de la SVD",  
x = expression(X[1]),  
y = expression(X[2])  
)+  
theme_krul()
```

Interpretación geométrica de la SVD

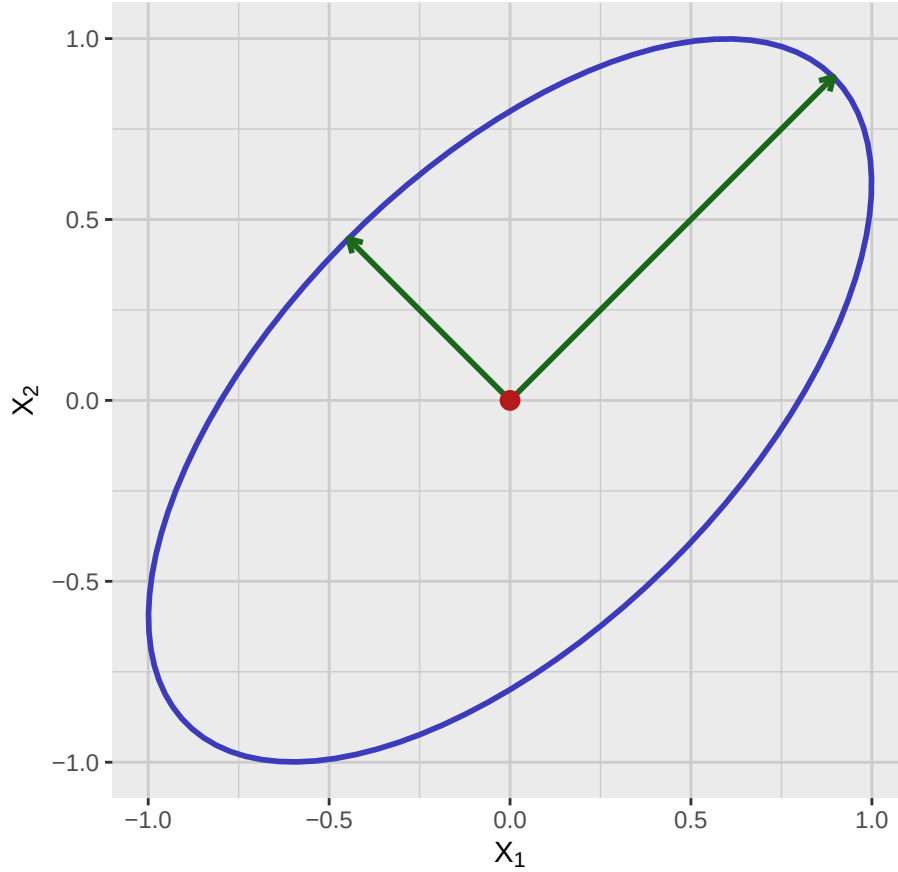


Figure 5.2: Interpretación geométrica de la SVD de una normal bivalente. El punto rojo es la media, las flechas verdes son los eigenvectores escalados por la raíz de los eigenvalores y la elipse azul es el contorno de distancia de Mahalanobis igual a 1.

5.5 Clausura bajo transformaciones lineales

Teorema 5.5.1 (Closure under Linear Transformations). *Let $X \sim \mathcal{N}_p(\mu, \Sigma)$ be a random vector with normal multivariate distribution. Then, for any matrix A of size $m \times p$ and any vector b of size m , the random vector*

$$Y = AX + b$$

also has a normal multivariate distribution, specifically $Y \sim \mathcal{N}_m(A\mu + b, A\Sigma A^T)$.

Comentario 5.5.2. This theorem states that if we apply a linear transformation to a random vector with a normal multivariate distribution, the resulting random vector will also have a normal multivariate distribution. The mean and covariance of the transformed vector are given by the formulas in the theorem. This property is particularly useful in applications such as regression analysis, where we often need to transform the data to fit a model while maintaining the normality of the residuals. Furthermore, it allows us to derive new distributions from existing ones, which is a key aspect of multivariate statistical theory. In practice, this theorem is often used to derive the

distribution of linear combinations of random variables, which is a common operation in multivariate statistics.

Ejemplo 5.5.3. Let $X \sim \mathcal{N}_2(\mu, \Sigma)$ be a random vector with mean $\mu = (1, 2)$ and covariance matrix

$$\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix}$$

Now, consider the linear transformation

$$Y = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} X + \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

In other words,

$$Y_1 = 2X_1 + X_2$$

$$Y_2 = 3X_2 + 1$$

Using the closure under linear transformations theorem, we can find the mean and covariance of Y :

$$\begin{aligned} \text{Mean: } A\mu + b &= \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \end{bmatrix} \\ \text{Covariance: } A\Sigma A^T &= \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 8 & 9 \\ 9 & 18 \end{bmatrix} \end{aligned}$$

Thus,

$$Y \sim \mathcal{N}_2 \left(\begin{bmatrix} 4 \\ 7 \end{bmatrix}, \begin{bmatrix} 8 & 9 \\ 9 & 18 \end{bmatrix} \right)$$

Comentario 5.5.4. Notice that since projection into a subspace is a linear transformation, the closure under linear transformations theorem implies that any subset of the components of a normal multivariate distribution also has a normal multivariate distribution. This is a very useful property, as it allows us to analyze subsets of variables while still maintaining the normality assumption.

5.6 Pruebas de normalidad multivariante

There are several statistical tests to check whether a given dataset follows a multivariate normal distribution. These tests are essential in multivariate statistics, as many methods assume normality. Here are some of the most commonly used tests, together with their implementation in R:

1. **Mardia's Test:** Based on multivariate skewness and kurtosis. For a multivariate normal distribution, skewness should be close to 0 and kurtosis close to $p(p+2)$, where p is the number of variables. Implemented in R via `MVN::mvn(your_data, mvn_test = "mardia")`.
2. **Henze-Zirkler (HZ) Test:** Uses the empirical characteristic function to measure deviations from normality. It has good global power and is suitable for moderate dimensions. Implemented in R via `MVN::mvn(your_data, mvn_test = "hz")`.

3. **Royston's Test:** A multivariate extension of the Shapiro-Wilk test, based on the product of univariate Shapiro-Wilk statistics applied to principal components. Easy to interpret, though power decreases in high dimensions. Implemented in R via `MVN::mvn(your_data, mvn_test = "royston")`.
4. **Doornik-Hansen Test:** Combines skewness and kurtosis into a single omnibus test, providing a robust alternative to Mardia. Implemented in R via `MVN::mvn(your_data, mvn_test = "dh")`.
5. **Graphical Methods (Chi-square Q-Q plots):** Plot the Mahalanobis distances of the data against the quantiles of a chi-square distribution. Deviations from a straight line indicate departures from multivariate normality. Implemented in R via `multivariate_diagnostic_plot(data = your_data, type = "qq")` .

Chapter 6

Análisis de componentes principales

El Análisis de Componentes Principales (PCA) es una técnica potente para reducir la dimensionalidad preservando la mayor información posible (medida por la varianza). Transforma las variables originales en un nuevo conjunto de variables no correlacionadas llamadas componentes principales, ordenadas por la varianza que explican.

6.1 Entendiendo la varianza total

Sea Σ la matriz de covarianzas del conjunto. La *varianza total* se da por la traza:

$$\text{Total Variance} = \text{Tr } \Sigma = \sum_{i=1}^p \Sigma_{ii},$$

donde Σ_{ii} son las varianzas muestrales de cada variable. Usando la descomposición en valores/vectores propios (SVD) de Σ , existen matrices Q y Λ tales que:

$$\Sigma = Q\Lambda Q^T,$$

siendo Λ diagonal con los valores propios $\lambda_1, \dots, \lambda_p$. Entonces la varianza total puede escribirse como:

$$\text{Total Variance} = \text{Tr } \Sigma = \text{Tr } Q\Lambda Q^T = \text{Tr } \Lambda = \lambda_1 + \dots + \lambda_p.$$

Las columnas de Q son las *direcciones de los componentes principales*; los valores de Λ indican cuánta varianza explica cada componente. El primero (mayor valor propio) capta la mayor varianza, el segundo la siguiente, y así sucesivamente. Estos componentes inducen un nuevo sistema de coordenadas; cada punto puede representarse por sus *puntuaciones* de componentes, relacionadas con las variables por:

$$X = Q(PC) + \bar{x}$$

and

$$PC = Q^T(X - \bar{x}) \tag{6.1}$$

Aquí X es el vector en coordenadas originales, PC el de puntuaciones, y $\bar{x}$ el vector de medias.

Para ilustrar estas ideas, consideramos el conjunto de la figura:

sample_data_plot

Datos de muestra

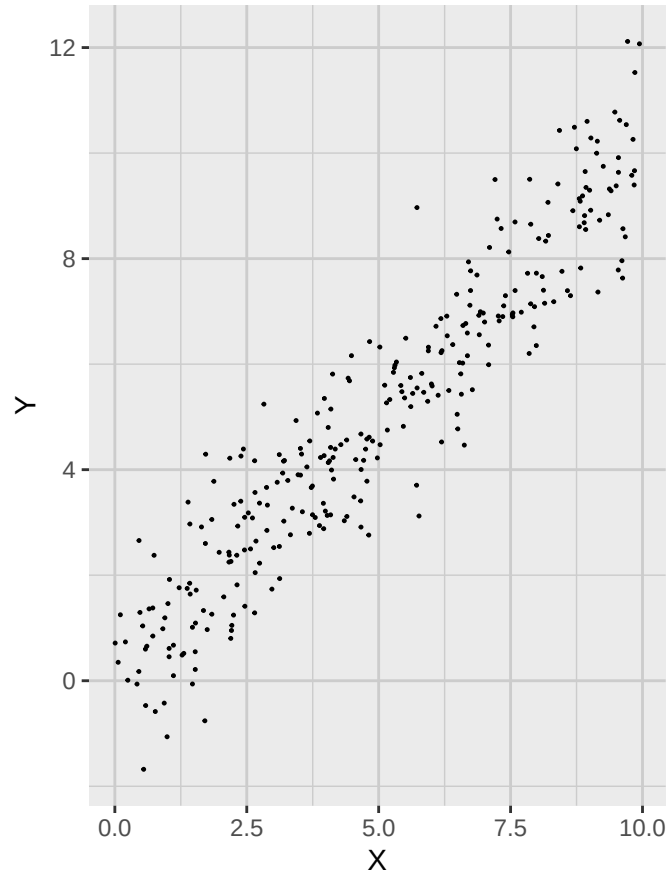


Figure 6.1: Datos de muestra con dos variables altamente correlacionadas.

Este conjunto consta de 300 observaciones de dos variables, X y Y : la primera es uniforme en $[0, 10]$ y la segunda es $Y = X + \epsilon$, con ϵ normal de media 0 y desviación 1. La matriz de covarianzas es:

```
Sigma <- cov(sample_data_tbl)
```

$$\Sigma = \begin{bmatrix} 7.874751 & 7.7745959 \\ 7.7745959 & 8.6510962 \end{bmatrix}$$

La varianza total es:

$$\text{Total Variance} = \text{Tr } \Sigma = \Sigma_{11} + \Sigma_{22} = 16.5258472$$

Consideramos ahora la descomposición espectral de Σ para obtener:

```
# Compute singular vectors and values
eig <- eigen(Sigma)
vectors <- eig$vectors
values <- eig$values
```

$$Q = \begin{bmatrix} 0.689251 & -0.7245227 \\ 0.7245227 & 0.689251 \end{bmatrix}$$

and

$$\Lambda = \begin{bmatrix} 16.0472039 & 0 \\ 0 & 0.4786433 \end{bmatrix}$$

Las direcciones de componentes principales son las columnas de Q y la varianza explicada por cada componente es el valor propio correspondiente. En particular, como el vector de medias es:

```
sample_mean_vector <- colMeans(sample_data_tbl)
```

$$\bar{x} = 4.9961185$$

$$\bar{y} = 5.019733$$

tenemos que la relación entre variables originales y coordenadas de componentes principales es:

$$X = 0.689251PC_1 + -0.7245227PC_2 + 4.9961185$$

$$Y = 0.7245227PC_1 + 0.689251PC_2 + 5.019733$$

and

$$PC_1 = 0.689251(X - 4.9961185) + 0.7245227(Y - 5.019733)$$

$$PC_2 = -0.7245227(X - 4.9961185) + 0.689251(Y - 5.019733)$$

Con estas fórmulas, obtenemos la proyección de los datos sobre el primer y segundo componente. Los resultados se muestran a continuación:

```
sample_data_matrix <- as.matrix(sample_data_tbl)

# Center the data
centered_sample_data <- scale(sample_data_matrix, center = TRUE, scale = FALSE)

# SVD
svd_result <- svd(centered_sample_data)
v1 <- svd_result$v[, 1]
v2 <- svd_result$v[, 2]

# Projection onto PC, in centered coordinates
proj_PC1 <- (centered_sample_data %*% v1) %*% t(v1)
proj_PC2 <- (centered_sample_data %*% v2) %*% t(v2)
```

```
# Shift back to original location
proj_coords1 <- proj_PC1 + attr(centered_sample_data, "scaled:center")
proj_coords2 <- proj_PC2 + attr(centered_sample_data, "scaled:center")

# Provide unique names before converting to tibble
colnames(proj_coords1) <- c("x_proj1", "y_proj1")
colnames(proj_coords2) <- c("x_proj2", "y_proj2")

proj_coords1_tbl <- as_tibble(proj_coords1)
proj_coords2_tbl <- as_tibble(proj_coords2)

# Bind with original data
compare_proj1_tbl <- bind_cols(sample_data_tbl, proj_coords1_tbl)
compare_proj2_tbl <- bind_cols(sample_data_tbl, proj_coords2_tbl)

# Generate plot corresponding to the first projection
compare_plot1 <- ggplot(compare_proj1_tbl) +
  geom_point(
    aes(x = x, y = y),
    color = "black",
    size = 0.2,
    alpha = 1
  ) +
  geom_point(
    aes(x = x_proj1, y = y_proj1),
    color = c_pal("C red"),
    size = 0.2,
    alpha = 1
  ) +
  labs(
    title = "Proyección en el primer componente principal",
    x = "X",
    y = "Y"
  ) +
  coord_fixed() +
  theme_krul()

# Generate plot corresponding to the second projection
compare_plot2 <- ggplot(compare_proj2_tbl) +
  geom_point(
    aes(x = x, y = y),
    color = "black",
    size = 0.2,
    alpha = 1
```

```
) +  
geom_point(  
  aes(x = x_proj2, y = y_proj2),  
  color = c_pal("C red"),  
  size = 0.2,  
  alpha = 1  
) +  
labs(  
  title = "Proyección en el segundo componente principal",  
  x = "X",  
  y = "Y"  
) +  
coord_fixed() +  
theme_krul()  
  
# Combine plots  
compare_plot1 + compare_plot2 +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Proyecciones de componentes principales",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
)  
)
```

Proyecciones de componentes principales

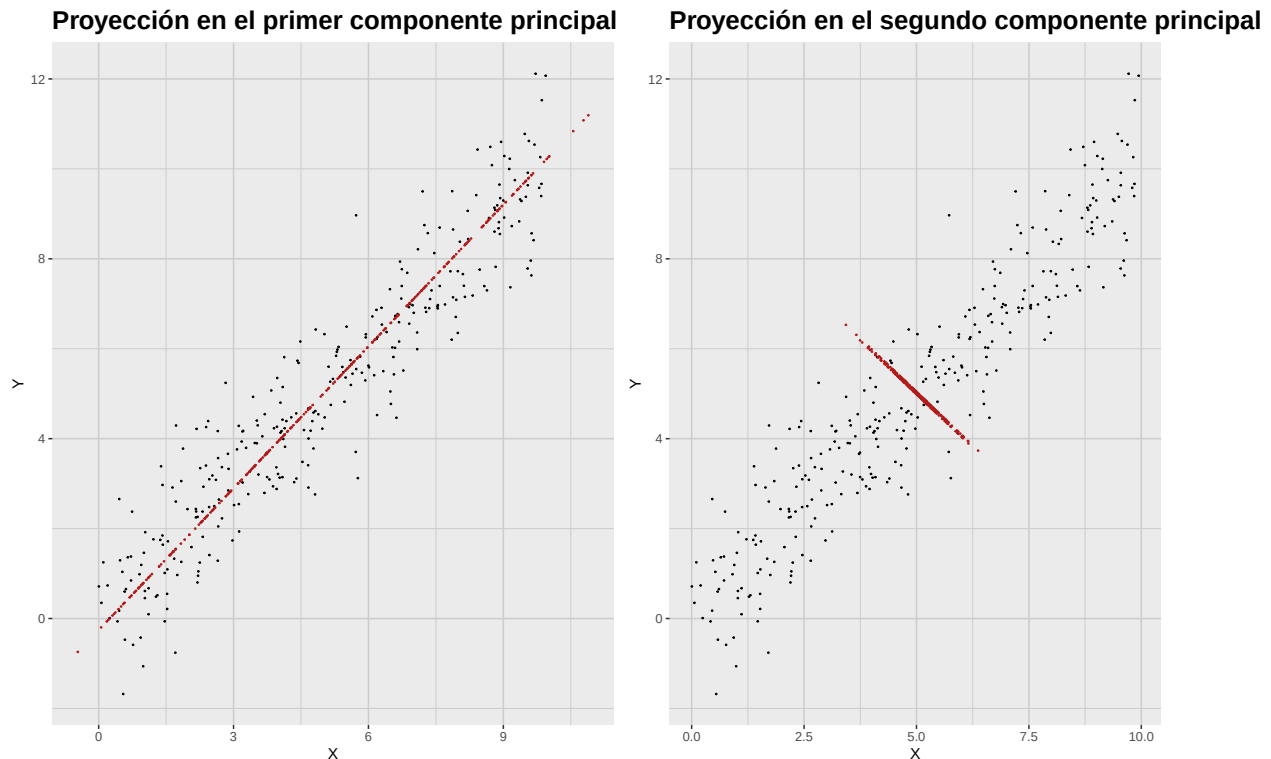


Figure 6.2: Proyecciones sobre las direcciones de los componentes principales. Se pierde mucha más información al proyectar sobre el segundo componente.

6.2 Varianza acumulada y gráficos scree

Como se indicó, el PCA se usa para reducción de dimensión. Buscamos reducir variables reteniendo la mayor información. Para decidir cuántos componentes conservar, examinamos la varianza explicada por cada uno. Es común usar un *gráfico scree*, que muestra valores propios en y y el índice del componente en x ; ayuda a identificar el “codo”. Otro gráfico útil es el de *varianza acumulada*, que muestra la proporción acumulada explicada. La proporción explicada por el componente k es:

$$\frac{\lambda_k}{\sum_{i=1}^p \lambda_i}$$

Este gráfico ayuda a determinar cuántos componentes se requieren para explicar, por ejemplo, 70% o 90% de la varianza total.

Para ilustrar, usaremos el conocido conjunto “iris”, con cuatro mediciones (largo/ancho de sépalo y pétalo) para tres especies (setosa, versicolor, virginica). Realizaremos PCA y construiremos el scree para visualizar la varianza explicada por cada componente. Primero, cargamos los datos y calculamos covarianzas por especie:

```
# Load the iris dataset
data(iris)
```

```
iris_setosa_tbl <- as_tibble(iris) %>%
  filter(Species == "setosa") %>%
  select(-Species)

iris_versicolor_tbl <- as_tibble(iris) %>%
  filter(Species == "versicolor") %>%
  select(-Species)

iris_virginica_tbl <- as_tibble(iris) %>%
  filter(Species == "virginica") %>%
  select(-Species)

# Compute covariance matrix
cov_matrix_setosa <- cov(iris_setosa_tbl)
cov_matrix_versicolor <- cov(iris_versicolor_tbl)
cov_matrix_virginica <- cov(iris_virginica_tbl)

# Singular value decomposition
eig_setosa <- eigen(cov_matrix_setosa)
eig_versicolor <- eigen(cov_matrix_versicolor)
eig_virginica <- eigen(cov_matrix_virginica)
```

The covariance matrices for each species are given by:

$$\Sigma_{setosa} = \begin{bmatrix} 0.12 & 0.1 & 0.02 & 0.01 \\ 0.1 & 0.14 & 0.01 & 0.01 \\ 0.02 & 0.01 & 0.03 & 0.01 \\ 0.01 & 0.01 & 0.01 & 0.01 \end{bmatrix}$$

$$\Sigma_{versicolor} = \begin{bmatrix} 0.27 & 0.09 & 0.18 & 0.06 \\ 0.09 & 0.1 & 0.08 & 0.04 \\ 0.18 & 0.08 & 0.22 & 0.07 \\ 0.06 & 0.04 & 0.07 & 0.04 \end{bmatrix}$$

$$\Sigma_{virginica} = \begin{bmatrix} 0.4 & 0.09 & 0.3 & 0.05 \\ 0.09 & 0.1 & 0.07 & 0.05 \\ 0.3 & 0.07 & 0.3 & 0.05 \\ 0.05 & 0.05 & 0.05 & 0.08 \end{bmatrix}$$

The singular values for each species are given by:

$$\Lambda_{setosa} = \begin{bmatrix} 0.24 & 0 & 0 & 0 \\ 0 & 0.04 & 0 & 0 \\ 0 & 0 & 0.03 & 0 \\ 0 & 0 & 0 & 0.01 \end{bmatrix}$$

$$\Lambda_{versicolor} = \begin{bmatrix} 0.49 & 0 & 0 & 0 \\ 0 & 0.07 & 0 & 0 \\ 0 & 0 & 0.05 & 0 \\ 0 & 0 & 0 & 0.01 \end{bmatrix}$$

$$\Lambda_{virginica} = \begin{bmatrix} 0.7 & 0 & 0 & 0 \\ 0 & 0.11 & 0 & 0 \\ 0 & 0 & 0.05 & 0 \\ 0 & 0 & 0 & 0.03 \end{bmatrix}$$

The associated principal component directions are given by the columns of the matrices

$$Q_{setosa} = \begin{bmatrix} -0.67 & 0.6 & 0.44 & -0.04 \\ -0.73 & -0.62 & -0.27 & -0.02 \\ -0.1 & 0.49 & -0.83 & -0.24 \\ -0.06 & 0.13 & -0.2 & 0.97 \end{bmatrix}$$

$$Q_{versicolor} = \begin{bmatrix} -0.69 & 0.67 & -0.27 & 0.1 \\ -0.31 & -0.57 & -0.73 & -0.23 \\ -0.62 & -0.34 & 0.63 & -0.32 \\ -0.21 & -0.34 & 0.06 & 0.92 \end{bmatrix}$$

$$Q_{virginica} = \begin{bmatrix} -0.74 & -0.17 & -0.53 & 0.37 \\ -0.2 & 0.75 & -0.33 & -0.54 \\ -0.63 & -0.17 & 0.65 & -0.39 \\ -0.12 & 0.62 & 0.43 & 0.65 \end{bmatrix}$$

The scree plot and cumulative variance plot for the three species are shown in the following figure:

```
# Scree plot data
scree_data <- tibble(
  Component = 1:4,
  Setosa = eig_setosa$values,
  Versicolor = eig_versicolor$values,
  Virginica = eig_virginica$values
) %>%
  pivot_longer(
    cols = -Component,
    names_to = "Species",
    values_to = "SingularValue"
  )
# Scree plot
scree_plot <- scree_data %>%
```

```
ggplot(aes(x = Component, y = SingularValue, color = Species)) +
  geom_point(size = 3) +
  geom_line() +
  c_scale_color("C red", "C blue", "C green") +
  scale_x_continuous(breaks = 1:4) +
  labs(
    title = "Gráfico scree del conjunto Iris",
    x = "Componente principal",
    y = "Valor singular"
  ) +
  theme_krul() +
  theme(legend.position = "top")

# Cumulative variance data
cumulative_variance_data <- scree_data %>%
  group_by(Species) %>%
  arrange(Component) %>%
  mutate(CumulativeVariance = cumsum(SingularValue) / sum(SingularValue))

# Cumulative variance plot
cumulative_variance_plot <- cumulative_variance_data %>%
  ggplot(aes(x = Component, y = CumulativeVariance, color = Species)) +
  geom_point(size = 3) +
  geom_line() +
  c_scale_color("C red", "C blue", "C green") +
  scale_x_continuous(breaks = 1:4) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(
    title = "Varianza acumulada explicada por los componentes principales",
    x = "Componente principal",
    y = "Varianza acumulada explicada"
  ) +
  theme_krul() +
  theme(legend.position = "top")

# Combine scree plot and cumulative variance plot
scree_plot + cumulative_variance_plot +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Gráfico scree y varianza acumulada para el conjunto Iris",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
```

)

Gráfico scree y varianza acumulada para el conjunto Iris

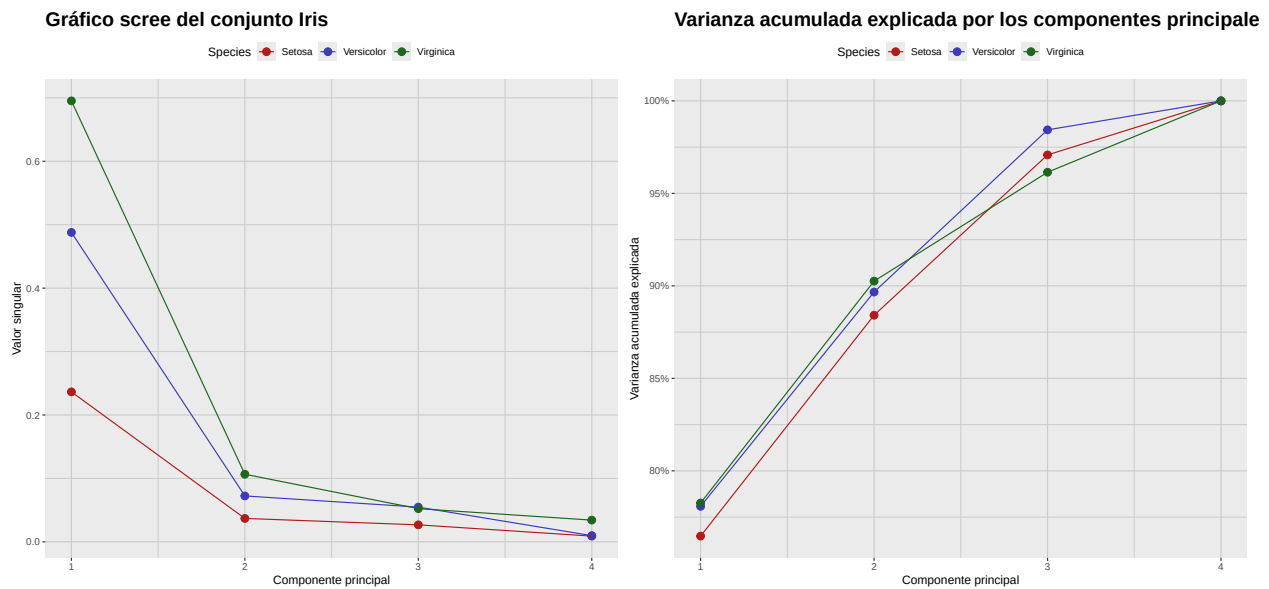


Figure 6.3: Principal Component Analysis of the Iris Dataset by Species. To decide the number of components to keep, we can use the scree plot (left) and look for the ‘elbow’ where the explained variance levels off. Alternatively, we can use the cumulative variance plot (right) to determine how many components are needed to explain a desired amount of total variance, such as 70% or 90%.

6.3 Cargas (loadings) y biplots

Recall that equation (6.1) gives the relation between the original variables and the principal component scores. Using this equation, we can quickly compute these scores for each observation in our dataset. The coefficients appearing in this equation are known as the *loadings* of the principal components, and they indicate how much each original variable contributes to each principal component. A *biplot* is a graphical representation that displays both the principal component scores of the observations and the loadings of the variables on the same plot. This allows us to visualize how the original variables contribute to the principal components and how the observations are distributed in the space defined by these components.

To illustrate this concept, we will create biplots for the first two principal components of the “iris” dataset for each species. We start by finding the loadings corresponding to each species, for which

we simply take the first two columns of the matrices Q_{setosa} , $Q_{versicolor}$, and $Q_{virginica}$ shown above.

$$\text{Loadings}_{setosa} = \begin{bmatrix} -0.67 & 0.6 \\ -0.73 & -0.62 \\ -0.1 & 0.49 \\ -0.06 & 0.13 \end{bmatrix}$$

$$\text{Loadings}_{versicolor} = \begin{bmatrix} -0.69 & 0.67 \\ -0.31 & -0.57 \\ -0.62 & -0.34 \\ -0.21 & -0.34 \end{bmatrix}$$

$$\text{Loadings}_{virginica} = \begin{bmatrix} -0.74 & -0.17 \\ -0.2 & 0.75 \\ -0.63 & -0.17 \\ -0.12 & 0.62 \end{bmatrix}$$

We can then compute the principal component scores for each observation by simply multiplying the values of our original variables by these matrices. The resulting biplots for each species are shown in the following figure:

```
# Function to prepare biplot data from precomputed singular vectors
prepare_biplot_from_eigen <- function(tbl, eig) {
  data_matrix <- as.matrix(tbl)
  centered_data_matrix <- scale(data_matrix, center = TRUE, scale = FALSE)

  # Scores: project observations onto PC1 and PC2
  scores <- centered_data_matrix %*% eig$vectors[, 1:2]
  colnames(scores) <- c("PC1", "PC2")
  scores_tbl <- as_tibble(scores) %>%
    mutate(Obs = row_number())

  # Loadings: first two singular vectors
  loadings <- eig$vectors[, 1:2]
  colnames(loadings) <- c("PC1", "PC2")
  loadings <- as_tibble(loadings) %>%
    mutate(Variable = colnames(tbl))

  list(scores = scores_tbl, loadings = loadings)
}

# Prepare data for each species
setosa_data <- prepare_biplot_from_eigen(
  iris_setosa_tbl,
  eig_setosa
)
```

```
versicolor_data <- prepare_biplot_from_eigen(
  iris_versicolor_tbl,
  eig_versicolor
)

virginica_data <- prepare_biplot_from_eigen(
  iris_virginica_tbl,
  eig_virginica
)

# Function to create biplot
create_biplot <- function(scores_tbl, loadings_tbl, species_name) {
  ggplot() +
    geom_point(
      data = scores_tbl,
      aes(x = PC1, y = PC2),
      color = c_pal("C red"),
      alpha = 1,
      size = 0.3
    ) +
    geom_segment(
      data = loadings_tbl,
      aes(
        x = 0,
        y = 0,
        xend = PC1 * max(scores_tbl$PC1),
        yend = PC2 * max(scores_tbl$PC2)
      ),
      arrow = arrow(length = unit(0.3, "cm")),
      color = c_pal("C blue"),
    ) +
    geom_text(
      data = loadings_tbl,
      aes(
        x = PC1 * max(scores_tbl$PC1) * 1.1,
        y = PC2 * max(scores_tbl$PC2) * 1.1,
        label = Variable
      ),
      color = "black",
      size = 5
    ) +
    labs(
      title = paste("Biplot para", species_name),
      x = "PC1",
```

```
    y = "PC2"
  ) +
  theme_minimal()
}

# Create biplots
biplot_setosa <- create_biplot(
  setosa_data$scores,
  setosa_data$loadings,
  "Setosa"
)

biplot_versicolor <- create_biplot(
  versicolor_data$scores,
  versicolor_data$loadings,
  "Versicolor"
)

biplot_virginica <- create_biplot(
  virginica_data$scores,
  virginica_data$loadings,
  "Virginica"
)

# Combine biplots
biplot_setosa + biplot_versicolor + biplot_virginica +
  plot_layout(ncol = 3) +
  plot_annotation(
    title = "Biplots de especies de Iris (PC1 vs PC2)",
    theme = theme(
      plot.title = element_text(size = 24, face = "bold")
    )
  )
)
```

Biplots de especies de Iris (PC1 vs PC2)

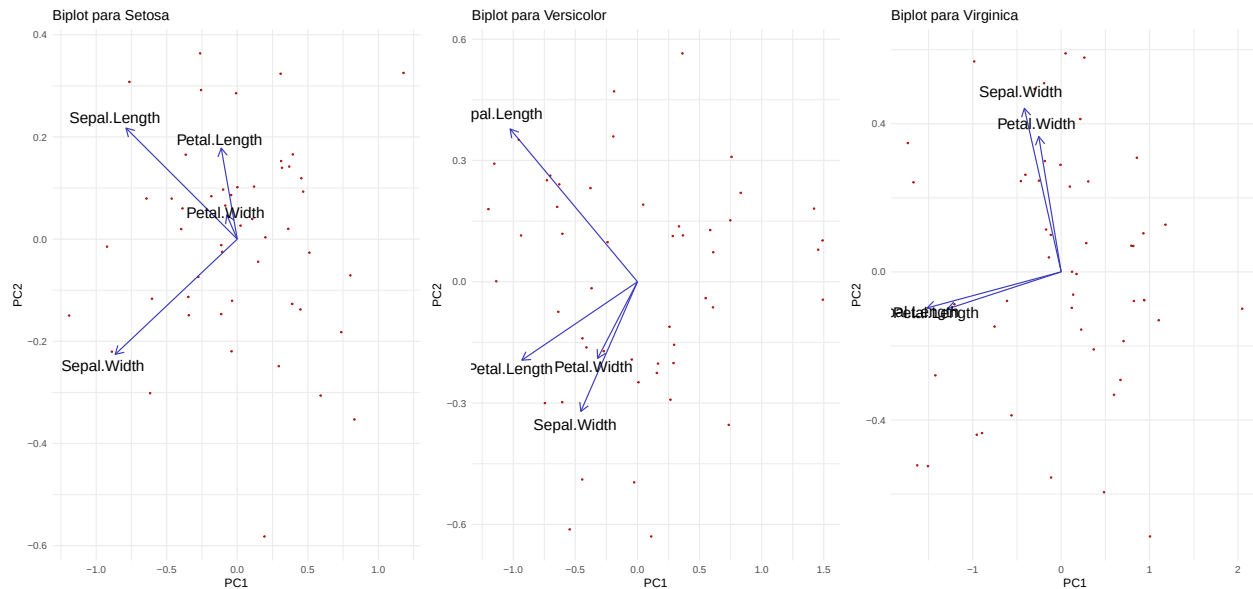


Figure 6.4: Cargas y puntuaciones para los dos primeros componentes principales del conjunto Iris por especie.

6.4 Resumen del proceso de PCA

In general, when performing PCA, it is recommended to follow these steps:

1. **Standardize the data (if needed).** It is not always easy to decide whether to standardize the data or not. Here are some guidelines:
 - If the variables are measured on different scales (e.g., height in cm and weight in kg), standardization is usually recommended.
 - If the variables are on the same scale and have similar variances, standardization may not be necessary.
 - If the variables have very different variances, standardization can help to give equal weight to all variables.
2. **Compute the covariance matrix.**
 - The matrix captures how variables vary together.
 - Its size is $p \times p$, where p is the number of variables.
3. **Find the singular value decomposition (SVD) of the covariance matrix.**
 - Eigenvalues measure the variance explained by each principal component.
 - Eigenvectors give the directions (linear combinations of variables) of the components.
4. **Rank the components.**

- Order the singular values from largest to smallest.
- The singular vector corresponding to the largest singular value is the first principal component.

5. Decide how many components to keep.

- *Kaiser criterion*: keep the singular values > 1 (for correlation matrix).
- *Scree plot*: look for the “elbow” where explained variance levels off.
- *Cumulative variance*: keep enough components to explain about 70% – 90% of the total variance.

6. Find the principal component scores and loadings.

- Project the original data onto the selected principal components.
- Use the formula: $PC = Q^T(X - \bar{x})$.

7. Interpret the components.

- Examine the *loadings* to see which variables contribute the most.
- Larger absolute values indicate stronger influence.
- Use biplots to visualize scores and loadings together.
- Apply dimension reduction before regression, clustering, or other analyses.
- Apply noise reduction by discarding low-variance components.

Part III

Análisis Factorial

Chapter 7

Análisis factorial: modelos teóricos

El *análisis factorial* es una técnica estadística ampliamente usada para identificar patrones ocultos o estructuras latentes detrás de grandes conjuntos de variables observadas. Busca reducir la complejidad agrupando variables relacionadas en un menor número de *factores* subyacentes, más fáciles de interpretar.

Algunas áreas de aplicación comunes incluyen:

- **Psychology and social sciences:** to uncover latent traits such as intelligence, anxiety, personality dimensions, or satisfaction levels, based on responses to questionnaires or surveys.
- **Marketing and consumer research:** to identify purchasing motives, brand perceptions, or market segments from customer behavior data.
- **Finance and economics:** to reveal hidden drivers of stock returns, macroeconomic indicators, or risk factors.
- **Education:** to analyze test items and determine whether they measure common skills or abilities.
- **Biological and health sciences:** to find latent structures in genetic data, symptom checklists, or patient-reported outcomes.

En la práctica se usa cuando se sospecha que unas cuantas dimensiones no observadas explican las correlaciones entre muchas variables. Es especialmente útil para construir escalas de medición, reducir datos y entender la estructura de conjuntos complejos.

7.1 Planteamiento del análisis factorial

The theoretical context for factor analysis is a relation of the form

$$X = \mu + LF + \epsilon$$

donde:

- X is a random vector of p observed variables (indicators).

- μ is a constant vector of dimension p .
- L is a $p \times m$ constant matrix known as the matrix of *factor loadings*.
- $F \sim \mathcal{N}(0, I_m)$ is a vector of m latent factors (unobserved variables) that are assumed to be normally distributed with mean zero and identity covariance matrix.
- $\epsilon \sim \mathcal{N}(0, \Psi)$ is a vector of unique errors (specific variances) that are also assumed to be normally distributed with mean zero and diagonal covariance matrix $\Psi = \text{diag}(\sigma_1^2, \dots, \sigma_p^2)$.
- The factors F and the errors ϵ are assumed to be independent, in particular $\text{Cov}[F, \epsilon] = 0$.

Como consecuencia de estos supuestos, se tiene

$$\mathbb{E}[X] = \mathbb{E}[\mu + LF + \epsilon] = \mathbb{E}[\mu] + L\mathbb{E}[F] + \mathbb{E}[\epsilon] = \mu$$

and

$$\begin{aligned} \text{Var}[X] &= \text{Var}[LF + \epsilon] \\ &= \mathbb{E}[(LF + \epsilon)(LF + \epsilon)^T] \\ &= \mathbb{E}[LFF^T L^T] + \mathbb{E}[\epsilon\epsilon^T] + \mathbb{E}[LF\epsilon^T] + \mathbb{E}[\epsilon F^T L^T] \\ &= L\mathbb{E}[FF^T]L^T + \mathbb{E}[\epsilon\epsilon^T] + L\mathbb{E}[F\epsilon^T] + \mathbb{E}[\epsilon F^T]L^T \\ &= LI_m L^T + \Psi + L0 + 0L^T \\ &= LL^T + \Psi \end{aligned}$$

Además, como X es combinación lineal de normales, se sigue que

$$X \sim \mathcal{N}(\mu, LL^T + \Psi)$$

7.2 Descomposición de varianza

La descomposición $\text{Var}[X] = LL^T + \Psi$ muestra que la varianza total de las variables observadas puede separarse en dos componentes:

- **Varianza común (LL^T):** explicada por los factores comunes F , capta la variabilidad compartida atribuible a la estructura latente.
- **Varianza única (Ψ):** específica de cada variable observada; incluye errores de medición y variabilidad propia.

In particular, the variance of each individual observed variable X_i can be expressed as

$$\text{Var}[X_i] = \sum_{j=1}^m L_{ij}^2 + \sigma_i^2$$

donde $h_i^2 = \sum_{j=1}^m L_{ij}^2$ es la *comunalidad* de X_i (parte explicada por factores comunes) y σ_i^2 la varianza única.

7.3 Estimación de puntuaciones factoriales

Suponga el modelo

$$X = \mu + LF + \epsilon$$

con las mismas condiciones. Entonces la distribución conjunta de X y F es

$$\begin{bmatrix} X \\ F \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu \\ 0 \end{bmatrix}, \begin{bmatrix} LL^T + \Psi & L \\ L^T & I_m \end{bmatrix} \right)$$

Nótese que, en particular,

$$\text{Cov}[X, F] = \mathbb{E}[(X - \mu)(F - 0)^T] = \mathbb{E}[(LF + \epsilon)F^T] = L\mathbb{E}[FF^T] + \mathbb{E}[\epsilon F^T] = LI_m + 0 = L$$

Usando propiedades de la normal multivariante, se obtiene la distribución condicional de F dado X , que también es normal con media y covarianza

$$[F|X = x] \sim \mathcal{N} \left(L^T(LL^T + \Psi)^{-1}(x - \mu), I_m - L^T(LL^T + \Psi)^{-1}L \right)$$

La media de esta distribución condicional,

$$\mathbb{E}[F|X = x] = L^T(LL^T + \Psi)^{-1}(x - \mu)$$

se conoce como la *puntuación factorial* para la observación $X = x$. Proporciona una estimación de los factores latentes a partir de las variables observadas y las cargas. La matriz de covarianzas

$$I_m - L^T(LL^T + \Psi)^{-1}L$$

representa la incertidumbre asociada a las puntuaciones estimadas.

Comentario 7.3.1. Notice that, if

$$f = \mathbb{E}[F|X = x] = L^T(LL^T + \Psi)^{-1}(x - \mu)$$

then, in general, the *residual*

$$r(x) = x - \mu - Lf$$

is different from zero. In fact, as a function of the random variable X , the residual has probability distribution

$$r(X) \sim \mathcal{N} \left(0, L \left(I_m + L^T \Psi^{-1} L \right)^{-1} L^T + \Psi \right)$$

7.4 Rotación de factores

Un aspecto importante del análisis factorial es que los factores no están definidos de manera única. Si F es un vector de factores, cualquier transformación ortogonal $F^* = QF$ (con $Q^T Q = I$) produce un nuevo conjunto de factores que explica la misma varianza. En particular,

$$\text{Var}[F^*] = \text{Var}[QF] = Q \text{Var}[F] Q^T = Q I_m Q^T = I_m$$

Las cargas correspondientes se transforman como $L^* = LQ^T$. En otras palabras, si

$$X = \mu + LF + \epsilon$$

is a valid factor model, then so is

$$X = \mu + L^*F^* + \epsilon = \mu + LQ^TQF + \epsilon = \mu + LF + \epsilon$$

Esto implica que la solución no es única y distintas rotaciones pueden conducir a interpretaciones diferentes. En el siguiente capítulo exploraremos métodos prácticos y técnicas de rotación para lograr soluciones más interpretables.

Chapter 8

Análisis factorial: métodos prácticos

Recordemos del capítulo anterior que el modelo clásico de análisis factorial es

$$X = \mu + LF + \epsilon$$

donde:

- X is a random vector of p observed variables (indicators).
- μ is a constant vector of dimension p .
- L is a $p \times m$ constant matrix known as the matrix of *factor loadings*.
- $F \sim \mathcal{N}(0, I_m)$ is a vector of m latent factors (unobserved variables) that are assumed to be normally distributed with mean zero and identity covariance matrix.
- $\epsilon \sim \mathcal{N}(0, \Psi)$ is a vector of unique errors (specific variances) that are also assumed to be normally distributed with mean zero and diagonal covariance matrix $\Psi = \text{diag}(\sigma_1^2, \dots, \sigma_p^2)$.
- The factors F and the errors ϵ are assumed to be independent, in particular $\text{Cov}[F, \epsilon] = 0$.

Nótese que, como $\epsilon \sim \mathcal{N}(0, \Psi)$, este modelo queda completamente determinado por μ , L y Ψ . En este capítulo discutiremos métodos prácticos para estimar estos parámetros a partir de datos y para obtener las puntuaciones factoriales de F una vez estimado el modelo.

8.1 Evaluación de la factorizabilidad

El primer paso es evaluar si los datos son adecuados para AF; es decir, si las variables observadas exhiben correlaciones suficientes que justifiquen un modelo factorial. Varias pruebas y medidas pueden emplearse:

1. **Revisión de la matriz de correlaciones:** inspección visual en busca de patrones altos (p.ej., > 0.3) que sugieran factores comunes.
2. **Prueba de esfericidad de Bartlett:** contrasta identidad; un resultado significativo ($p < 0.05$) sugiere correlación suficiente.

3. **Kaiser–Meyer–Olkin (KMO):** mide la proporción de varianza atribuible a factores comunes; valores cercanos a 1 indican utilidad (típicamente > 0.6 es aceptable).

8.2 Decidir el número de factores

Determinar cuántos factores retener es crucial y puede afectar los resultados e interpretaciones. Algunos métodos:

1. **Criterio de Kaiser:** retener valores singulares > 1 .
2. **Gráfico scree:** buscar el “codo”.
3. **Análisis paralelo:** comparar con datos aleatorios del mismo tamaño.
4. **Índices de ajuste:** (RMSEA, TLI) para evaluar modelos con distinto número de factores.

8.3 Extracción de factores

Una vez decidido el número de factores, el siguiente paso es extraerlos. Métodos disponibles:

1. **Ejes principales (PAF):** se centra en la varianza compartida; estima comunalidades y extrae factores desde la matriz reducida.
2. **Máxima verosimilitud (ML):** estima cargas maximizando la verosimilitud; permite pruebas e índices de ajuste.

8.4 Rotación de factores

Tras extraer factores, suele ser útil rotarlos para lograr una estructura más simple e interpretable. La rotación puede ser ortogonal (sin correlación) u oblicua (con correlación). Métodos comunes:

1. **Varimax Rotation:** An orthogonal rotation method that maximizes the variance of squared loadings within each factor, leading to a simpler structure where each variable loads highly on one factor and low on others.
2. **Quartimax Rotation:** Another orthogonal rotation method that simplifies the rows of the loading matrix, aiming to have each variable load highly on as few factors as possible.
3. **Equamax Rotation:** A hybrid orthogonal rotation method that combines the goals of varimax and quartimax, balancing the simplification of both rows and columns of the loading matrix.
4. **Promax Rotation:** An oblique rotation method that allows factors to correlate. It starts with a varimax rotation and then raises the loadings to a power to achieve a simpler structure.

8.5 Computing Factor Scores

Once the factor loadings have been estimated and the factors have been rotated (if applicable), the next step is to compute factor scores for each observation in the dataset. Factor scores represent

the estimated values of the latent factors for each individual based on their observed variable scores. Several methods can be used to compute factor scores:

1. **Regression Method:** This method estimates factor scores by regressing the observed variables on the estimated factor loadings. It provides unbiased estimates of factor scores but may not preserve the original variance of the factors.
2. **Bartlett Method:** This method computes factor scores by minimizing the residual variance of the observed variables. It tends to produce factor scores that are more reliable and have lower measurement error.
3. **Anderson-Rubin Method:** This method generates factor scores that are uncorrelated and standardized. It is particularly useful when the factors are assumed to be orthogonal.

8.6 Assessing Model Fit

After fitting a factor analysis model, it is important to assess how well the model fits the data. Several statistical tests and fit indices can be used to evaluate model fit:

1. **Chi-Square Test of Model Fit:** This test evaluates the null hypothesis that the model-implied covariance matrix matches the observed covariance matrix. A non-significant result ($p\text{-value} > 0.05$) suggests that the model fits the data well.
2. **Root Mean Square Error of Approximation (RMSEA):** This index measures the discrepancy between the observed and model-implied covariance matrices per degree of freedom. Values less than 0.06 indicate a good fit.
3. **Tucker-Lewis Index (TLI):** This index compares the fit of the specified model to a null model. Values greater than 0.95 indicate a good fit.
4. **Standardized Root Mean Square Residual (SRMR):** This index measures the average discrepancy between observed and predicted correlations. Values less than 0.08 indicate a good fit.

8.7 Visualizing the Model

Visualizing the results of a factor analysis can help in interpreting the factors and understanding the relationships among variables. Common visualization techniques include:

1. **Scree Plot:** A scree plot displays the singular values in descending order, helping to visualize the number of factors to retain.
2. **Factor Loading Plot:** A scatter plot of factor loadings can help visualize how variables load on different factors, especially after rotation.
3. **Biplot:** A biplot combines factor scores and loadings in a single plot, allowing for the visualization of both observations and variable relationships in the factor space.
4. **Heatmap of Loadings:** A heatmap can be used to visualize the magnitude of factor loadings across variables and factors, highlighting patterns and clusters.

Comentario 8.7.1. Notice that Factor Loading Plots and Biplots only make sense when we have 2 or 3 factors, otherwise we cannot visualize them properly.

8.8 Example: Factor Analysis of the Holzinger-Swineford Dataset

We will now provide an example of factor analysis using the “HolzingerSwineford1939” dataset from the “lavaan” package. This dataset contains scores from a battery of 9 cognitive tests administered to 301 students in grades 7 and 8. The tests were designed to measure 3 latent abilities: visual/spatial, verbal, and speed. For this example we will focus only on tests x4 to x9 so that we can fit a two-factor model and visualize the results in a biplot. The exact description of each test is given below:

1. **Paragraph (x4):** Reading comprehension. Students read short paragraphs and answer questions about the main idea, details, and simple inferences.
2. **Sentence (x5):** Sentence completion. Choose the word or phrase that best completes a sentence so it is clear and grammatical; emphasizes vocabulary-in-context and syntax.
3. **Word Meaning (x6):** Vocabulary. Select the best synonym/definition for a target word; reflects breadth of lexical knowledge.
4. **Addition (x7):** Timed arithmetic fluency. Complete many simple addition problems quickly and accurately; captures processing speed with minimal reasoning load.
5. **Counting Dots (x8):** Rapid visual enumeration. Quickly count small groups of dots across many items; measures visual scanning and speeded accuracy.
6. **Straight–Curved Capitals (x9):** Perceptual discrimination speed. Scan capital letters and mark those with only straight strokes (versus any curved strokes), under strict time limits.

To perform this analysis, we will start by loading the necessary libraries and the dataset:

```
# Load required libraries
library(lavaan)
library(psych)
library(scales)

# Load the Holzinger-Swineford1939 dataset
data(HolzingerSwineford1939)

# Select the relevant tests
hs_data <- as_tibble(HolzingerSwineford1939) %>%
  select("x4", "x5", "x6", "x7", "x8", "x9")
```

8.8.1 Assessing Factorability

We start our factor analysis process by assessing the factorability for tests x4–x9. First we collect all the required information:

```
# Correlation matrix (pairwise complete observations)
Sigma <- cor(hs_data)

# Bartlett's test and KMO measure
n_eff <- nrow(hs_data)
bart <- cortest.bartlett(Sigma, n = n_eff)
kmo <- KMO(Sigma)
```

Mostramos ahora el mapa de calor de correlaciones:

```
corr_df <- Sigma %>%
  as_tibble(rownames = "Var1") %>%
  pivot_longer(-Var1, names_to = "Var2", values_to = "r")

# Preserve matrix order on axes
corr_df$Var1 <- factor(corr_df$Var1, levels = colnames(Sigma))
corr_df$Var2 <- factor(corr_df$Var2, levels = colnames(Sigma))

ggplot(corr_df, aes(x = Var2, y = Var1, fill = r)) +
  geom_tile(color = "black") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 3) +
  scale_fill_gradient2(
    low = c_pal("C blue"), mid = "white", high = c_pal("C red"),
    midpoint = 0, limits = c(-1, 1)
  ) +
  coord_fixed() +
  labs(
    title = "Mapa de calor de correlaciones (x4-x9)",
    x = "", y = "", fill = "Correlación"
  ) +
  theme(
    plot.title = element_text(size = 18, face = "bold"),
    panel.background = element_blank(),
    panel.grid = element_blank(),
    axis.ticks = element_blank()
  )
```

Mapa de calor de correlaciones (x4–x9)

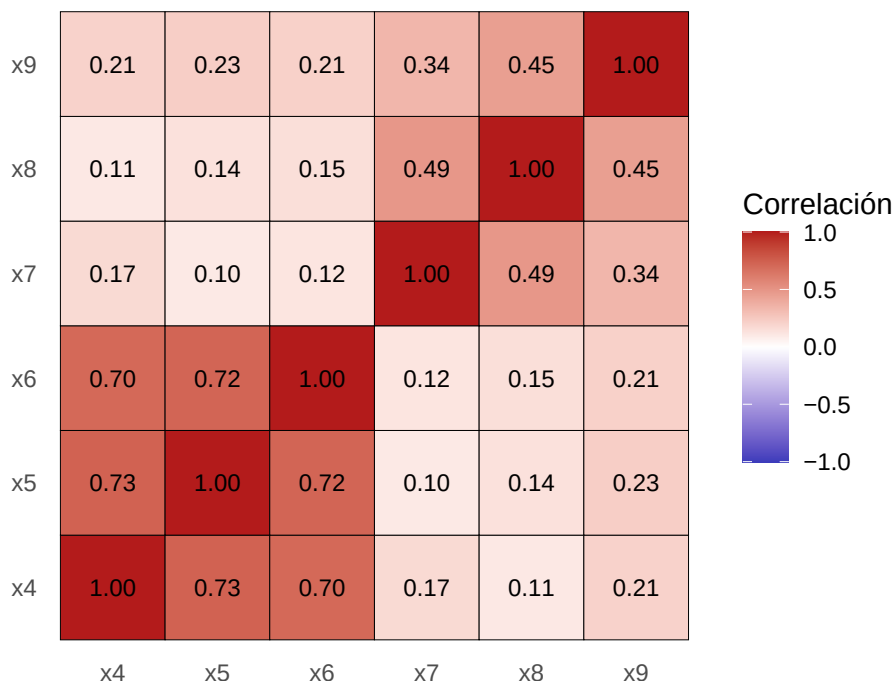


Figure 8.1: Mapa de calor de correlaciones para x4–x9. Se observan correlaciones fuertes entre las pruebas verbales (x4, x5, x6) y entre las de rapidez (x7, x8, x9), sugiriendo un modelo de dos factores.

Obsérvese la fuerte correlación entre pruebas verbales (x4, x5, x6) y entre rapidez (x7, x8, x9), pero no entre grupos. Esto sugiere que un modelo de dos factores es adecuado. Reportamos ahora Bartlett y KMO:

Bartlett's test of sphericity = 0
Kaiser-Meyer-Olkin measure = 0.7308774

Como se observa, todas las pruebas indican que el AF es apropiado para este conjunto.

8.8.2 Decidir el número de factores

Usaremos un gráfico scree y uno de varianza acumulada para decidir cuántos factores retener. Como usamos la matriz de correlaciones, buscamos valores singulares > 1 (Kaiser) y el “codo” en el scree.

```
# Scree analysis from the correlation matrix
eig <- eigen(Sigma)

scree_df <- tibble(
  component = seq_along(eig$values),
  singular_value = eig$values
) %>%
```

```
mutate(
  cumulative = cumsum(singular_value) / sum(singular_value)
)

# Scree plot
scree_plot <- scree_df %>%
  ggplot(aes(x = component, y = singular_value)) +
  geom_point(size = 3, color = c_pal("C red")) +
  geom_line(color = c_pal("C blue")) +
  geom_hline(
    yintercept = 1,
    linetype = "dashed",
    color = c_pal("C green"),
    linewidth = 1.5
  ) +
  scale_x_continuous(breaks = scree_df$component) +
  labs(
    title = "Gráfico scree",
    x = "Componente principal",
    y = "Valor singular"
  ) +
  theme_krul()

# Cumulative variance plot
cum_plot <- ggplot(scree_df, aes(x = component, y = cumulative)) +
  geom_point(size = 3, color = c_pal("C red")) +
  geom_line(color = c_pal("C blue")) +
  scale_x_continuous(breaks = scree_df$component) +
  scale_y_continuous(
    labels = percent_format(accuracy = 1),
    limits = c(0, 1)
  ) +
  labs(
    title = "Varianza acumulada",
    x = "Componente principal",
    y = "Varianza acumulada explicada"
  ) +
  theme_krul()

# Combine scree plot and cumulative variance plot
scree_plot + cum_plot + plot_layout(ncol = 2) +
  plot_annotation(
    title = paste(
      "Gráfico scree y varianza acumulada para",
      "el conjunto Holzinger-Swineford"
    )
  )
```

```
),
  theme = theme(
    plot.title = element_text(
      size = 24,
      face = "bold"
    )
  )
)
```

Gráfico scree y varianza acumulada para el conjunto Holzinger–Swineford

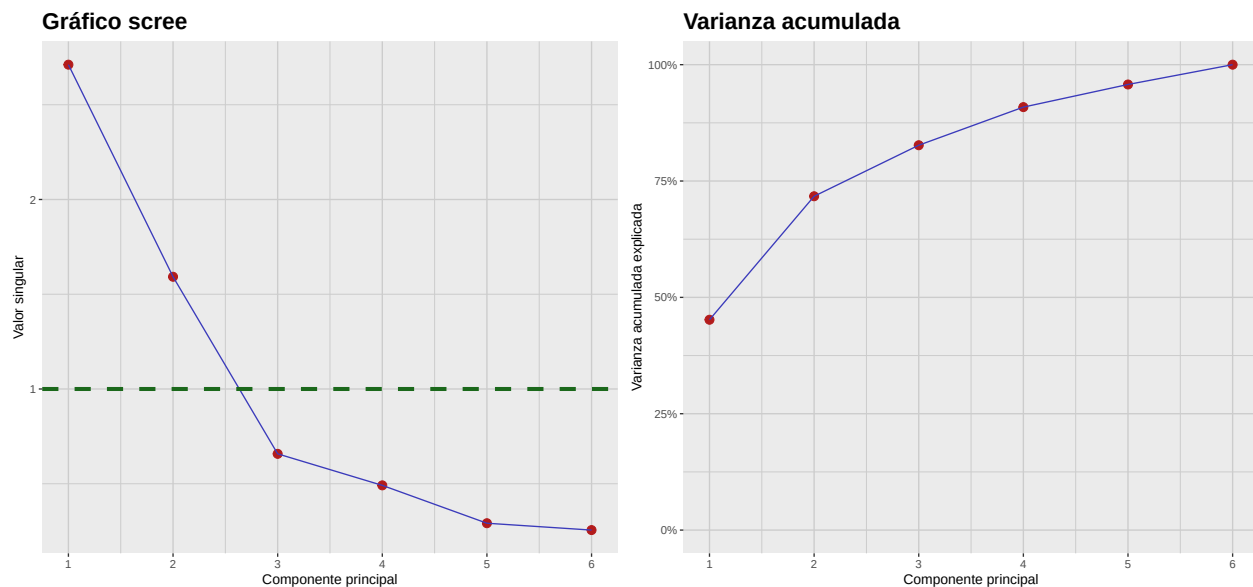


Figure 8.2: Scree plot and cumulative variance for tests x4–x9. Both plots suggest that a two-factor solution is appropriate for this dataset.

Notice that both the scree plot and the cumulative variance plot suggest that a two-factor solution is appropriate for analysing the six tests included in this dataset.

8.8.3 Extracting, Rotating, and Scoring Factors

Even though extracting, rotating and scoring factors are theoretically separate steps, in practice they are often performed together using specialized software or statistical packages. Here we will use the `psych::fa` function to perform all three steps at once. We will fit a 2-factor model using maximum likelihood (ML) estimation, compute factor scores using the regression method, and perform a varimax rotation to simplify the factor structure.

```
# Fit 2-factor FA using ML, regression scores and varimax rotation
fa_rot <- fa(
  r = hs_data,
  nfactors = 2,
  rotate = "varimax",
```

```

  fm = "ml",
  scores = "regression",
  use = "pairwise"
)

# Extract factor loadings into a tibble
loadings <- fa_rot$loadings[, 1:2] %>%
  unclass() %>%
  as_tibble(rownames = "Variable") %>%
  rename(F1 = 2, F2 = 3)
# Extract factor scores into a tibble
scores <- fa_rot$scores %>%
  as_tibble() %>%
  setNames(c("F1", "F2")) %>%
  mutate(Obs = row_number())

```

8.8.4 Assessing Model Fit

We now evaluate how well the 2-factor ML solution reproduces the observed correlations. For this, we perform the tests χ -squared, RMSEA, CFI, TLI, and RMSR.

```

# Extract standard fit indices from psych::fa object (ML)
chisq <- fa_rot$STATISTIC
df    <- fa_rot$dof
pval  <- fa_rot$PVAL
rmsea <- as.numeric(fa_rot$RMSEA[1])
tli   <- fa_rot$TLI
bic   <- fa_rot$BIC

L    <- as.matrix(fa_rot$loadings)
Phi  <- if (!is.null(fa_rot$Phi)) fa_rot$Phi else diag(ncol(L))
uni  <- if (!is.null(fa_rot$uniquenesses)) {
  fa_rot$uniquenesses
} else {
  1 - rowSums((L %*% Phi) * L)
}
Rhat <- L %*% Phi %*% t(L) + diag(as.numeric(uni))

# Helper function to extract lower-triangle entries (off-diagonal)
lt_offdiag <- function(M) M[lower.tri(M, diag = FALSE)]

# Classic RMSR
res <- Sigma - Rhat
rmsr <- sqrt(mean(lt_offdiag(res)^2, na.rm = TRUE))

# Standardized RMSR (SRMR)

```

```
is_cov <- any(abs(diag(Sigma) - 1) > 1e-8)
S_obs_cor <- if (is_cov) stats::cov2cor(Sigma) else Sigma
S_hat_cor <- if (is_cov) stats::cov2cor(Rhat) else Rhat

res_cor <- S_obs_cor - S_hat_cor

# SRMR = sqrt( sum_{i<j}(res_ij^2) / sum_{i<j}(obs_ij^2) )
num <- sum(lt_offdiag(res_cor)^2, na.rm = TRUE)
den <- sum(lt_offdiag(S_obs_cor)^2, na.rm = TRUE)
srmr <- sqrt(num / den)
```

The results of these tests are:

$$\begin{aligned}\chi^2 \text{ Test} &= 0.1156716 \\ \text{RMSEA} &= 0.0531269 \\ \text{TLI} &= 0.9804573 \\ \text{RMSR} &= 0.0411927\end{aligned}$$

All tests indicate that the 2-factor model fits the data well.

8.8.5 Visualizing the Model

Finally, we will visualize the model using a biplot. We will also rotate the factors manually to illustrate what could happen if we did not choose an appropriate rotation method.

```
# Scaling factor for arrows (keep consistent across plots)
arrow_scale <- 3

# Base (unrotated) biplot
p_base <- ggplot() +
  geom_point(
    data = scores,
    aes(x = F1, y = F2),
    alpha = 1,
    color = c_pal("C red"),
    size = 0.5
  ) +
  geom_segment(
    data = loadings,
    aes(x = 0, y = 0, xend = F1 * arrow_scale, yend = F2 * arrow_scale),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C blue"),
    linewidth = 1.2
  ) +
  geom_text(
    data = loadings,
```

```
    aes(x = F1 * arrow_scale, y = F2 * arrow_scale, label = Variable),
    color = "black",
    hjust = -0.5
  ) +
  labs(
    title = "Varimax Rotation",
    x = "Factor 1",
    y = "Factor 2"
  ) +
  theme_krul()

# 45° rotation of scores and loadings
theta <- pi / 4
cos_t <- cos(theta)
sin_t <- sin(theta)

scores_rot <- scores %>%
  mutate(
    F1_rot = F1 * cos_t - F2 * sin_t,
    F2_rot = F1 * sin_t + F2 * cos_t
  )

loadings_rot <- loadings %>%
  mutate(
    F1_rot = F1 * cos_t - F2 * sin_t,
    F2_rot = F1 * sin_t + F2 * cos_t
  )

# Rotated biplot
p_rot <- ggplot() +
  geom_point(
    data = scores_rot,
    aes(x = F1_rot, y = F2_rot),
    alpha = 1,
    color = c_pal("C red"),
    size = 0.5
  ) +
  geom_segment(
    data = loadings_rot,
    aes(x = 0, y = 0, xend = F1_rot * arrow_scale, yend = F2_rot * arrow_scale),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C blue"),
    linewidth = 1.2
  ) +
  geom_text(
```

```
data = loadings_rot,
aes(x = F1_rot * arrow_scale, y = F2_rot * arrow_scale, label = Variable),
color = "black",
hjust = -0.5
) +
labs(
  title = "Manual Rotation",
  x = "Rotated Factor 1",
  y = "Rotated Factor 2"
) +
theme_krul()

# Optional: share axis limits for fair comparison
xr <- range(
  c(
    scores$F1,
    scores_rot$F1_rot,
    loadings$F1 * arrow_scale,
    loadings_rot$F1_rot * arrow_scale
  ),
  na.rm = TRUE
)

yr <- range(
  c(
    scores$F2,
    scores_rot$F2_rot,
    loadings$F2 * arrow_scale,
    loadings_rot$F2_rot * arrow_scale
  ),
  na.rm = TRUE
)

p_base <- p_base + coord_equal(xlim = xr, ylim = yr)
p_rot <- p_rot + coord_equal(xlim = xr, ylim = yr)

# Side-by-side using patchwork
p_base + p_rot + plot_layout(ncol = 2) +
  plot_annotation(
    title = paste(
      "Biplots of the factor analysis model",
      "using Varimax and Manual Rotations"
    ),
    theme = theme(
      plot.title = element_text(
        size = 24,
```

```

    face = "bold"
  )
)
)

```

Biplots of the factor analysis model using Varimax and Manual Rotations

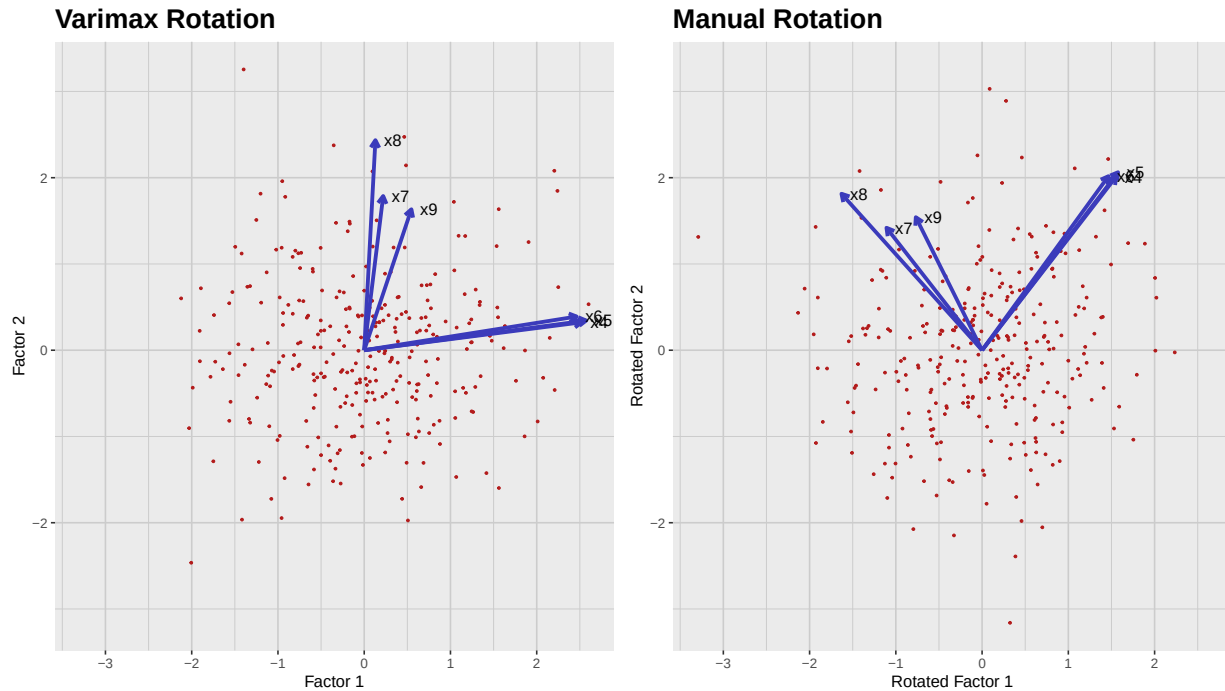


Figure 8.3: Biplots of the 2-factor solution for tests x4–x9. Left: Varimax rotation. Right: Manual 45° rotation. Notice how the varimax rotation leads to a simpler and more interpretable structure, with verbal tests (x4, x5, x6) loading highly on Factor 1 and speed tests (x7, x8, x9) loading highly on Factor 2.

